Unicorn Systemspecifikation för Replit 3.0

Komplett Implementationsguide för Klusterhantering och Vidareutveckling

Version: 1.0
Datum: 2026-01-02
Syfte: Ge Replit 3.0 (AI-utvecklingsassistent) fullständig kontext och implementationsvägledning för vidareutveckling av Unicorn-systemet med fokus på klusterhantering enligt Mats vision.

Innehållsförteckning

1. Nulägesanalys
2. Mats Vision för Klusterhantering
3. Överenskommen Design
4. Gap-analys
5. Implementationsguide för Replit 3.0
6. Viktiga Principer och Regler

---

1. Nulägesanalys

1.1 Systemöversikt

Unicorn är en AI-driven planeringsplattform för fältserviceföretag på den nordiska marknaden, utvecklad i samarbete med Kinab AB. Plattformen optimerar ruttplanering, resursallokering, ekonomisk kontroll, produktivitet och prediktiv analys. Målet är att först bevisa värde med Kinab AB inom avfallshantering och sophämtning, sedan skala till en kommersiell multi-tenant SaaS-lösning för alla nordiska serviceföretag.

Status: Funktionell prototyp med modern webbstack, nordisk estetik och AI-integration.

1.2 Teknisk Stack

Frontend

- **Framework:** React med TypeScript
- **Build Tool:** Vite
- **UI-bibliotek:** shadcn/ui
- **Styling:** Tailwind CSS (antas baserat på shadcn/ui)
- **Font:** Inter för UI
- **Tema:** Dark/light mode support
- **Språk:** Svenska (sv) för användargränssnitt

Backend

- **Runtime:** Node.js

- **Framework:** Express.js
- **Språk:** TypeScript

Databas

- **DBMS:** PostgreSQL
- **ORM:** Drizzle ORM
- **Multi-tenancy:** Stöd på databasnivå

Kartfunktionalitet

- **Kartbibliotek:** react-leaflet för interaktiva kartvisualiseringar
- **Ruttoptimering:** OpenRouteService för ruttvisualisering
- **Geokodning:** OpenStreetMap Nominatim API för svenska adresser

AI-integration

- **Provider:** OpenAI via Replit AI Integrations
- **Modeller:**
- `gpt-4o-mini` för snabba analyser
- `gpt-4o` för komplexa beslut
- **Teknik:** Kontextdriven promptning med strukturerad JSON-output
- **Väderdata:** Open-Meteo API för 7-dagars prognoser (AI auto-scheduling)

Externa Tjänster

- **Unicorn Optimization Service:** Dedikerad ruttoptimeringstjänst (extern)
- **DataClean Service:** Extern tjänst för datavalidering och geokodning
- **Modus 2.0:** Källa för CSV-data import

Kommunikation

- **WebSocket:** Token-baserad autentisering för realtidsnotifikationer
- **MCP Server (Model Context Protocol):** SSE och meddelandeendpoints för externa AI-assistenter

1.3 Befintlig Funktionalitet

UI/UX-komponenter

- **TopNav:** Sticky navigering med dropdown-kategorier
- Grunddata
- Planering
- Analys
- System
- **Global sökning:** Sökfunktionalitet över hela systemet
- **Mobilnavigering:** Hamburger-meny (Sheet-komponent)
- **Floating Action Button (FAB):** För snabba åtgärder
- **QuickStats:** Dashboard-kort med nyckeltal
- **WeekPlanner:** Drag-and-drop veckoplanering
- **RouteMap:** Visualisering av rutter
- **ObjectCards:** Kortvyer för objekt
- **Dashboard:** Omfattande översiktsvyer

Mobilgränssnitt för Fältteknikker

- **MobileFieldApp:** Stor knappstorlekvariant

- **SignatureCapture:** Digital signering
- **MaterialLog:** Autocomplete för nordiska avfallshanteringsobjekt
- **JobProtocolGenerator:** Automatisk PDF-rapportering

Datamodeller och Koncept

Kluster

- **Nuvarande status:** Grundläggande koncept implementerat
- **Visualisering:** Listor och interaktiva kartor
- **Geografisk organisering:** Kärnelement i systemet
- **Hierarkisk struktur:** `Område → Fastighet → Rum` (Area → Property → Room)
- Information ärvs nedåt i hierarkin
- Används för att hantera kundlokationer

Orderhantering

- **Arbetsflöde:** Omfattande med statusspårning
- **Ekonomi:** Kostnads- och värdespårning
- **Simulering:** Möjlighet att simulera ordrar
- **Livscykel:** Olika statusar för orderspårning

Prissystem

- **Tre-nivå hierarki:**
  1. Generell prislista
  2. Kundspecifik prislista
  3. Rabattbrev
- **Tillämpning:** För artiklar och tjänster

Flottan och Resurshantering

- **Fordon:** Spårning av fordonsflottan
- **Utrustning:** Utrustningshantering
- **Resursallokering:** Med kompetenser
- **Tillgänglighetsschemaläggning:** Planering av resurstillgänglighet

Prenumerationshantering

- **Återkommande tjänster:** Stöd för prenumerationer
- **Automatisk ordergenerering:** Baserat på prenumerationer

Planeringsparametrar

- **SLA-nivåer:** Konfiguration av servicenivåavtal
- **Tidsluckor:** Hantering av tidsfönster

AI-funktioner (Implementerade)

AICard - Kontextmedvetna AI-paneler

- **Placering:** På varje sida i systemet
- **WeekPlanner:**
- Auto-schemaläggning
- Arbetsbelastningsanalys
- Ruttoptimering
- **Andra moduler:** Kompakta insikter
- **Global AI-chatt:** Tillgänglig överallt

AI Planning Assistant

- **Veckoplanering:** Analyserar och optimerar
- **Arbetsbelastning:** Balanserar arbetsbelastning
- **Körtid:** Optimerar restid
- **Prioritering:** Prioriterar brådskande ordrar

AI Auto-Scheduling med Väderoptimering

- **Väderprognos:** Använder 7-dagars prognoser från Open-Meteo API
- **Kapacitetsjustering:** Anpassar efter väderförhållanden
- **Utomhusarbete:** Prioriterar vid gynnsamt väder
- **Klustergruppering:** Grupperar ordrar efter kluster

Realtidsfunktionalitet

WebSocket-notifikationer

- **Autentisering:** Token-baserad
- **Målgrupp:** Fältarbetare
- **Händelsetyper:**
- Jobbstatusändringar
- Schemaändringar
- Prioritetsändringar
- Anomalivarningar
- **Broadcast:** Systemövergripande meddelanden

GPS-positionsspårning

- **Realtid:** Spårning av fältresursers position
- **Breadcrumb trails:** Historik av positioner
- **Lagring:** `resourcePositions`-tabell
- **Uppdatering:** Via WebSocket med HTTP-fallback

Automatisk Anomaliövervakning

- **Bakgrundsjobb:** Upptäcker operativa avvikelser
- **Anomalityper:**
- Inaktuella positioner
- Försenade ordrar
- Avvikelser i inställningstid
- **Notifikationer:** Broadcast av hög/kritisk prioritet

Import och Datahantering

Modus 2.0 Import System

- **Dedikerad sida:** `/import`
- **Format:** CSV-import
- **Funktioner:**
- Datatransformering
- Validering

Address Search/Autocomplete

- **Provider:** OpenStreetMap Nominatim API
- **Fokus:** Svenska adresser
- **Geokodning:** Automatisk konvertering till koordinater

API:er och Endpoints

Mobile API Endpoints

- **Mobil inloggning:** Autentisering för mobilappar
- **Resurs-/orderhämtning:** Hämta data för fältarbetare
- **Statusuppdateringar:** Uppdatera orderstatus från fältet
- **Anteckningar:** Möjlighet att lägga till noteringar

MCP Server Endpoints

- **SSE (Server-Sent Events):** För strömning av händelser
- **Meddelandeendpoints:** För resurs- och arbetsorderhantering
- **Extern AI-integration:** Möjliggör interaktion för externa AI-assistenter

1.4 Systemdesignval

AI-First Approach

- **Kärnprincip:** AI-integration är fundamental
- **Övervägande:** Varje funktion utvärderas för AI-förbättring
- **Genomgående:** AI-funktioner integrerade i hela plattformen

Extern Optimering

- **Ruttoptimering:** Utlagd till separat Unicorn Optimization Service
- **Intern förberedelse:** Data förbereds internt innan optimering
- **Separation of Concerns:** Optimeringslogik separerad från kärnplattformen

Datavalidering

- **DataClean Service:** Hanterar extern datavalidering
- **Geokodning:** Verifiering och berikning av adressdata
- **Kvalitetssäkring:** Säkerställer datakvalitet före import

Lazy Object Loading

- **Prestandaoptimering:** För att hantera stora datamängder
- **Hooks:**
- `useObjectsByIds` - Hämta specifika objekt
- `useObjectSearch` - Sök bland objekt
- **Användarupplevelse:** Snabbare laddningstider

1.5 Vad Fungerar Bra

✅ Stark Grund

- Modern teknisk stack med TypeScript
- Komponentbaserad arkitektur (React + shadcn/ui)
- Robust databashantering (PostgreSQL + Drizzle ORM)
- Multi-tenancy support redan implementerad

✅ AI-Integration

- Fungerande AI-funktioner för planering
- Kontextmedvetna AI-paneler
- Väderbaserad optimering
- Strukturerad AI-integration via OpenAI

✅ Realtidsfunktioner

- WebSocket-kommunikation för notifikationer
- GPS-spårning av fältresurser
- Automatisk anomalidetektering

✅ Mobilstöd

- Dedikerade mobilendpoints
- Mobilanpassade komponenter
- Offline-kapacitet (antas via PWA-funktionalitet)

✅ Dataimport

- Fungerande Modus 2.0 import
- CSV-hantering med validering

---

2. Mats Vision för Klusterhantering

2.1 Kärnkoncept

Mats vision bygger på att kluster ska vara en **hierarkisk organisationsstruktur** som speglar kundens verkliga organisation och fysiska verklighet. Systemet ska möjliggöra intelligent **nedåtpropagering av information** där data definieras på rätt nivå och automatiskt blir tillgänglig för alla undernivåer.

2.2 Klusterhierarki - Organisatorisk och Fysisk

Principen

Kluster skapas med både **organisatoriska** och **fysiska** objekt som tillsammans bygger en komplett bild av kundens struktur.

Exempel på hierarki:

```
Castor Förvaltning AB (Koncern - Organisatoriskt)
└── BRF Huset (Bostadsrättsförening - Organisatoriskt)
    └── Fastighet Grytan 5 (Fastighet - Fysiskt)
        └── Återvinningsrum Södra (Rum - Fysiskt)
            ├── Matavfallskärl (Objekt - Fysiskt)
            │   ├── Kärl 1
            │   ├── Kärl 2
            │   └── Kärl 3
            └── Restavfallskärl (Objekt - Fysiskt)
                ├── Kärl 1
                ├── Kärl 2
                └── ... (8 kärl totalt)
```

Objekttyper i Hierarkin

- **Koncern/Organisation** (översta nivån)
- **Juridisk person** (t.ex. BRF, företag)
- **Fastighet** (fysisk plats)
- **Område/Rum** (del av fastighet)
- **Objekt** (t.ex. kärl, utrustning)
- **Individuella enheter** (specifika kärl, maskiner)

2.3 Kluster Speglar Kundens Verklighet

Princip 2: Kluster ska försöka återspegla kundens organisation och verkligheten så nära som möjligt.

Varför?
- Underlättar kommunikation med kunden
- Gör systemet intuitivt att använda
- Möjliggör rapportering som matchar kundens syn på sin verksamhet
- Förenklar problemlösning och support

Exempel:
Om Castor Förvaltning AB har 5 BRF:er och varje BRF har 3-10 fastigheter, ska klusterstrukturen exakt reflektera denna organisering.

2.4 Kundkoppling i Klustret

Princip 3: Kunden med fakturamottagare och kontakter kopplas in i klustret. Detta är grunden för:
- **Avisering:** Automatiska notifikationer till rätt kontaktpersoner
- **Kundinloggning:** Kunder kan logga in och se sin specifika information
- **Kommunikation:** Strukturerad kontakthantering

Datatyper som kopplas:
- Kundinformation (namn, organisationsnummer)
- Fakturamottagare (e-post, adress)
- Kontaktpersoner (namn, telefon, e-post, roll)
- Inloggningsuppgifter för kundportal

2.5 Nedåtpropagering av Information

Princip 4: Geografiska Positioner

Regel: Geografiska positioner definieras i klustret och **information faller nedåt i klustrets hierarki**.

Exempel:

```
Fastighet Grytan 5
├── Adress: Storgatan 248, Storstad
├── Koordinater: 59.1234, 18.5678
└── Denna information "faller ned" till:
    ├── Återvinningsrum Södra (ärver adress och koordinater)
    └── Alla kärl i rummet (ärver adress och koordinater)
```

Praktisk tillämpning:
När en fältarbetare ska utföra arbete på ett specifikt kärl, behöver adressen inte registreras på kärlnivå - den hämtas automatiskt från fastigheten ovanför.

Princip 5: Övrig Data med Nedåtfall

Regel: Övrig data definieras i klustret och **information faller nedåt om den inte spärras på nivå/objekt**.

Datatyper som faller nedåt:
- Prislista
- Rabattbrev
- Avtalsinformation

- Tillgänglighetstider
- Kontaktuppgifter
- Nyckelkoder
- Särskilda instruktioner

Visuellt från presentation:

```
Typ 1, Typ 2, Typ 3, Typ 4, Typ 5, Typ 6, Typ 7
Antal 2, Antal 4, Antal 2, Antal 1, Antal 1, Antal 2

Information från översta nivån faller ned till alla undernivåer
såvida den inte blockeras/överskrivs på en specifik nivå
```

Spärrmekanism:
Om en undernivå har **egen** information av samma typ, överskrider den information som faller uppifrån. Detta möjliggör undantag och specialfall.

Exempel:
- Castor Förvaltning AB: Prislista 120 (gäller alla)
- BRF Huset: [ingen egen prislista] → använder Prislista 120
- Fastighet Grytan 5: Prislista 125 (specialavtal) → **spärr**, använder 125 istället
- Alla rum och kärl under Fastighet Grytan 5: använder Prislista 125

2.6 Felanmälningar och Avvikelser

Princip 7: Övrig data som felanmälningar eller andra avvikelser **kopplas in i klustret** för att kunna:
- **Sökas:** Hitta alla felanmälningar för en specifik kund/fastighet
- **Skapa uppgifter:** Automatiskt generera arbetsordrar från felanmälningar

Exempel:

```
Felanmälan #1234
├── Kopplad till: Fastighet Grytan 5 → Återvinningsrum Södra → Matavfallskärl 2
├── Beskrivning: "Kärlet är skadat"
└── Genererar automatiskt:
    └── Arbetsorder: "Reparera eller byt ut Matavfallskärl 2"
```

2.7 Orderkoncept - Intelligent Orderbyggare

Princip 8: Orderkoncept Skapas och Pekas In

Arbetsflöde:
1. Ett **orderkoncept** (beställning) skapas
2. Orderkonceptet **pekas in i klustret** på önskad nivå/objekt
3. Orderkonceptet kan vara:
- **Generellt:** Gäller alla objekt av en viss typ
- **Specifikt:** Gäller endast specifika objekt
4. Artiklar/ordrar kan ha syfte att:
- **Multiplicera sig:** Skapa flera uppgifter baserat på antal objekt
- **Haka på objekt:** Fästa vid objekt som matchar kriterier

Exempel från presentationen:

```
Orderkoncept: "Tvätta alla matavfallskärl"
Pekas mot: BRF Huset
Resultat: Systemet hittar alla matavfallskärl under BRF Huset
         och skapar en uppgift för varje kärl
```

Princip 9: Orderkoncept Söker Sig Ned

Regel: Orderkoncept/beställning **söker sig ned i klustret** för att försöka hitta objekt att haka fast på och skapa en uppgift.

Söklogik:
1. Orderkoncept identifierar sin **måltyp** (t.ex. “matavfallskärl”)
2. Söker från den nivå den pekats mot
3. Traverserar nedåt i hierarkin
4. Identifierar alla matchande objekt
5. Skapar uppgift för varje matchning

Visuellt exempel:

```
Orderkoncept: T100 - Kärltvätt (matavfallskärl)
Pekat mot: Castor Förvaltning AB
│
├─ Söker i BRF Huset
│  └─ Söker i Fastighet Grytan 5
│     └─ Söker i Återvinningsrum Södra
│        └─ Hittar: 3 matavfallskärl → Skapar 3 uppgifter
└─ Söker i BRF Villa
   └─ Söker i Fastighet Lunden 12
      └─ Söker i Återvinningsrum Nord
         └─ Hittar: 5 matavfallskärl → Skapar 5 uppgifter

Total: 8 uppgifter skapade automatiskt
```

Princip 10: När Artikel Hittar Objekt, Skapas Uppgift

Regel: När artikeln hittat objekt att haka fast på, **skapas en uppgift** och övrig information för uppgiften länkas på från klustret och objektet.

Information som samlas:
- **Från objekt:** Typ, antal, specifika egenskaper
- **Från överordnade nivåer:** Plats, tillgång (nyckelkod), tider, mm
- **Från prissystem:** Pris, kostnad, produktionstid, leveranstid
- **Från artikel:** Beskrivning, standardutförandetid, resurskrav

Avisering:
När uppgift skapas, skickas automatiskt avisering till kontakter enligt klusterhierarkin.

Princip 11: Uppgiften Hämtar Prisinformation

Regel: Uppgiften hämtar information från:
- **Prislista**
- **Rabattbrev**

För att bestämma:
- Försäljningspris
- Kostnad

- Produktionstid
- Leveranstid

Prioordning:
1. Rabattbrev (om tillämpligt)
2. Kundspecifik prislista (om tillämplig)
3. Generell prislista

Princip 12: Orderkoncept Söker Igenom Hela Klustret

Regel: Orderkonceptet söker sig igenom hela klustret som det pekats emot och **uppgifter/artiklar på tjänster och varor skapas** baserat på objekt som hittats och den data som finns.

Beroendeartiklar:
Vissa artiklar skapar automatiskt **beroendeartiklar** som en del i strukturartiklar.

Exempel:
- **Huvudartikel:** T100 - Kärltvätt
- **Beroendeartiklar:**
- Förplock (förberedelsearbete)
- Avisering (notifiering till kund)
- Efterkontroll (kvalitetssäkring)

Resultat:
När en huvudartikel hittar ett matchande objekt, skapas automatiskt alla tillhörande beroendeartiklar som egna uppgifter i systemet.

2.8 Planeringsprocessen - “Snöret”

Koncept: “Snöret”

Definition: En metod att kombinera och optimera tre tidsintressen:
1. **Objektets eller kundens tider**
2. **Uppgiftens eller orderns tider**
3. **Utförarens eller teamets tider och tillgänglighet**

Visuellt från Slide 14:

```
Uppgift ──┐
          ├─→ SNÖRET ─→ Optimal Schemaläggning
Objekt ───┤
          │
Resurs ───┘
```

Komponenter:

Objektet/Kunden:
- Önskad leveranstid
- Tidsfönster (t.ex. “08:00-12:00”)
- Schema (återkommande tider)

Uppgiften/Ordern:
- Önskad leveranstid
- Avrop (när arbetet begärts)
- Leveranstid (deadline)
- Produktionstid (hur lång tid arbetet tar)

Utföraren/Teamet:
- Restid (till/från arbetsplats)
- Förtid (förberedelse)
- Administration
- Rast
- Nattvila
- Andra åtaganden (“Tandläkaren”)
- Team-beroenden

Tidskategorier i “Snöret”

Från presentationens Slide 15, exempel vecka 46 måndag:

Status	Uppdrag	Typ	Tid	Plats	Plan	Utfall
Utfört	Ledig	Egentid	2024-10-27 18:00	Söderby	36h	36h
Utfört	Förorder Kinab	Förb	2024-10-29 07:00	Järna	0,1	0
Aktiv	Inställelseresa	Auto	2024-10-29 07:10	-	2,4	2,3
Planerad	BRF Lövet Objekt 1	Prod	2024-10-30 09:40	Skövde	1,2	-
Planerad	Föravisering tel	Förb	2024-10-30 11:00	-	-	-
Planerad	Resa	Auto	2024-10-30 10:50	-	-	-

Tidstyper:
- **Egentid:** Utförarens egen tid (ledighet, vila)
- **Förb (Förberedelse):** Förberedelsearbete innan produktivt arbete
- **Auto (Automatiskt):** Systemgenererad tid (resor)
- **Prod (Produktion):** Själva utförandet av arbetet

Nyckeltal från “Snöret”:
- Procent av veckomål: 25%
- Kalkylerad arbetstid totalt: 39,5h
- Producerad tid: 8,5h
- Restid till/från uppdragsort: 5,2h
- Beläggning: 54%

Definition från Slide 15:

> “Snöret för en utförare eller team eller företaget består av egentid, förberedelsetid, restid, produktionstid, planerad eller den tid det tar/utfall”

2.9 Orderkonceptets Livscykel (Steg 1-11)

Från Slide 13 i presentationen:

Steg 1: Orderkoncept Skapas

- Definierar **vad som ska göras**
- Specificerar **vilka typer objekt det berör**

Steg 2: Orderkoncept Pekas In

- Pekas mot **enskilt objekt** eller **helt/del av kluster**
- **Önskad leveranstid** eller **leveransperiod** anges

Steg 3: Koppling mot Prislista och Kund

- Orderkoncept kopplas mot **vald prislista** och **kund**
- Artiklar får:
- Försäljningspris
- Kostnad
- Produktionstid

Steg 4: Artiklar Hakar Fast

- Orderkoncepts artiklar hakar fast på objekt efter typ
- Definierar utförandedata från prislista och objekt:
- Antal
- Plats
- Tid
- Pris
- Beroendeuppgifter
- **Avisering med orderbekräftelse** skickas

Steg 5: Total Uppgiftslista Skapas

- En total lista på uppgifter skapas
- Uppdateras löpande
- Utförderas kontinuerligt

Steg 6: Planering - Grov

- Definierar:
- **Vart?** (Plats)
- **När?** (Tid - möjlig)
- **Hur/Vem?** (Resurs/metod)
- **Hur länge?** (Tidsåtgång)
- **Vad?** (Typ, antal)
- **Värde?** (Ekonomiskt värde)

Steg 7: Planering - Fin

- **Tid - vecka:** Specificera exakt vecka
- **Plats - resa:** Ruttplanering
- **Tillgänglighet:** Kontrollera resurstillgänglighet
- **Tilldelad:** Tilldela specifik resurs
- **Planerad:** Bekräfta planering

- **Avisering:** Notifiera berörda parter

Steg 8: Utförande

- **Tid - vecka:** Aktuell vecka
- **Plats - resa:** Följ planerad rutt
- **Uppdatering art och antal:** Rapportera faktiskt utfört
- **Kontroll:** Kvalitetskontroll
- **Avisering:** Notifiera om utförande

Steg 9: Utfört

- **Kontrollerat:** Verifierat slutfört
- **Avisering:** Slutlig notifiering
- **Fakturering:** Underlag för faktura skapas

---

3. Överenskommen Design

3.1 Familjeträdsmetaforen

Den överenskomna designen använder **familjeträdsmetaforen** för att förklara klusterhantering:

> "I ett släktträd 'ärvs' egenskaper och relationer nedåt – farföräldrar → föräldrar → barn → barnbarn. På samma sätt fungerar Unicorns klustersystem enligt Mats vision: information registreras på en nivå och kan sedan användas automatiskt av alla nivåer under."

3.2 Tre Typer av Metadata

3.2.1 Fast Information (Låst på Nivån)

Koncept: Som en stämpel som bara sätts på en nivå. Den ändras inte automatiskt, men alla nivåer under kan se den när ordern byggs.

Egenskaper:
- Registreras på specifik nivå
- Ändras INTE automatiskt
- Synlig för alla undernivåer
- Används vid orderbygge

Exempel:
- **Castor-nivå:** "Alla fakturor ska till ekonomi@castor.se"
- **Fastighet-nivå:** "Nyckelkod 100 gäller för DENNA fastighet"
- **Kärl-nivå:** "Detta specifika kärl är skadat sedan 2024-11-15"

Användning:
Denna information stannar där den är – men "barnen" i trädet kan se den och använda den när de behöver.

3.2.2 Fallande Information (Ärvs Nedåt som Tyngdkraft)

Koncept: Som vatten som rinner från toppen och fyller alla nivåer under.

Egenskaper:
- Definieras på en nivå
- Ärvs automatiskt till alla undernivåer

- Ingen manuell kopiering behövs
- Uppdateras automatiskt vid ändring på ursprungsnivån

Exempel:

```
Castor Förvaltning AB
├── Avtal: Ramavtal 2024 ────┐
                             │ FALLER NED
                             ↓
├── BRF Huset
│   ├── Ärvt: Ramavtal 2024 ┐
│                           │ FALLER NED
│                           ↓
│   └── Fastighet Grytan 5
│       ├── Ärvt: Ramavtal 2024 ┐
│                               │ FALLER NED
│                               ↓
│       └── Återvinningsrum Södra
│           ├── Ärvt: Ramavtal 2024 ┐
│                                   │ FALLER NED
│                                   ↓
│           └── Alla kärl
│               └── Ärvt: Ramavtal 2024
```

Resultat:
När du skapar en order på ett enda kärl, vet systemet automatiskt vilket avtal som gäller.

3.2.3 Dynamisk Information (Ändras över Tid och Fortsätter Falla)

Koncept: Som levande vatten som kan ändra färg men fortsätter rinna nedåt.

Egenskaper:
- Kan uppdateras på ursprungsnivån
- Alla undernivåer får automatiskt den nya informationen
- Ingen manuell uppdatering på varje objekt
- Spårbar ändringshistorik (rekommenderat)

Exempel:

```
Tidpunkt 1:
BRF Huset: "Ska kontaktas våren 2026 för förnyelse"
└── Alla undernivåer ser: "Ska kontaktas våren 2026"

Tidpunkt 2:
BRF Huset uppdateras till: "Avtalet förnyas - ny kontakt våren 2027"
└── Alla undernivåer ser nu: "Avtalet förnyas - ny kontakt våren 2027"
```

Fördelar:
- Uppdatera EN gång
- Påverkar ALLA relaterade objekt
- Konsistent information överallt

3.3 Artiklar med Fasthakningar

Koncept: Artiklar som “Krokar”

Princip: Artiklar (tjänster och varor) har fasthakningar – som små krokar – som talar om var i trädet de hör hemma.

Fasthakningstyper:
- “Jag passar på KÄRL-nivå”
- “Jag passar på RUM-nivå”
- “Jag passar på KOD-nivå”
- “Jag passar på KÄRLMAT-nivå” (specifik kärltyp)

Artikel-till-Objekt Mappning

Artikel	Beskrivning	Fasthakning	Var den hamnar
T100	Kärltvätt standard	KÄRL	På varje kärl i systemet
T110	Rumstvätt	RUM	På återvinningsrum
KOD10	Öppna med kod	KOD	På fastigheter med kod
K100	Matavfallsdekal	KÄRLMAT	Bara på matavfall-skärl

Matchningslogik:
1. Artikel definierar sin fasthakningstyp
2. Orderkoncept pekas mot del av klustret
3. System traverserar klustret
4. För varje objekt:
- Kontrollera om objektets typ matchar artikelns fasthakning
- Om match: Skapa uppgift
- Om ingen match: Fortsätt söka

3.4 Automatisk Orderbyggare - “Bästa Orderunderlaget”

Scenario från Dokumentet

Uppgift: Johan ska tvätta matavfallskärl på Fastighet Grytan 5.

Systemets arbetsprocess:
Systemet bygger automatiskt ordern genom att läsa trädet uppifrån och nedåt.

ORDER #4521 - Komplett Exempel

VAR?
- **Data:** Storgatan 248, Storstad – Fastighet Grytan 5 – Återvinningsrum Södra
- **Källa:** FASTIGHET-nivå

TILLGÅNG?
- **Data:** Nyckelkod: 100
- **Källa:** FASTIGHET-nivå

HITTA FRAM?
- **Data:**
- Foto: dörr_och_fasad.jpg
- Foto: rum_och_karl.jpg
- **Källa:** RUM-nivå

VAD SKA GÖRAS?
- **Data:**
- Artikel: T100 – Kärltvätt standard
- Objekt: Matavfallskärl
- Antal: 3 st
- Typ: Standard 240L
- **Källa:** KÄRL-nivå

TID & RESURS?
- **Data:**
- Utförandetid: 3 min/kärl × 3 = 9 min
- Utförare: Johan (RBK Norrland)
- Fordon: AD0234
- **Källa:** ARTIKEL + TEAM-databas

FAKTURERING?
- **Data:**
- Kund: Castor Förvaltning AB
- Fakturareferens: ekonomi@castor.se
- Avtal: Ramavtal 2024
- Pris: Enligt prislista 120
- **Källa:** CASTOR-nivå

KONTAKT VID PROBLEM?
- **Data:**
- BRF Huset: 078-234 56 78
- Castor: 070-123 45 67
- **Källa:** BRF + CASTOR-nivå

Resultat

> "Johan fick allt han behöver – automatiskt!"

3.5 Genialiteten i Implementationen

Fördel 1: Registrera EN GÅNG – Använd ÖVERALLT

Exempel:
- Skriver in "Fakturareferens: ekonomi@castor.se" **en gång** på Castor-nivå
- Alla 200 kärl under Castor får **automatiskt** denna information i sina ordrar
- **Ingen** manuell kopiering
- **Ingen** risk för inkonsistens

Fördel 2: Ändra EN GÅNG - Uppdatera ÖVERALLT

Exempel:
- Castor byter fakturamejl till "faktura@castor.se"
- Ändra på CASTOR-nivå
- **Alla framtida ordrar** får det nya mejlet automatiskt
- **Inga** manuella uppdateringar på 200 kärl

Fördel 3: Intelligent Orderbyggare

Process:
1. Systemet "tittar uppåt" i trädet
2. Samlar all relevant information från varje nivå

3. Systemet “tittar på sin nivå”
4. Ser vad som ska göras
5. **Resultat:** Komplett order utan manuell datainsamling

Fördel 4: Skalbarhet

Exempel:
- Lägg till 50 nya kärl under BRF Huset
- De får **automatiskt** all information från nivåerna ovanför
- Artiklar som passar kärl “faller ner” **automatiskt**
- **Ingen** konfiguration per kärl behövs

3.6 Sammanfattning av Överenskommen Filosofi

> “Information ska registreras på rätt nivå och sedan falla nedåt som tyngdkraft. Artiklar ska hitta rätt plats som ett pussel. Ordern byggs automatiskt genom att systemet läser hela trädet uppifrån och nedåt.”

Resultat av Implementationen

✅ Mindre dubbelregistrering
✅ Färre fel
✅ Snabbare orderbygge
✅ Bättre orderunderlag för utförare
✅ Automatisk fakturering med rätt info
✅ Skalbart när verksamheten växer

---

4. Gap-analys

4.1 Nuvarande Klusterfunktionalitet vs. Målbild

Nuläge

- ✅ Grundläggande klusterkoncept implementerat
- ✅ Hierarkisk struktur: `Område → Fastighet → Rum`
- ✅ Visualisering med listor och kartor
- ⚠️ **SAKNAS:** Fullständig nedåtpropageringsmekanism
- ⚠️ **SAKNAS:** Organisatorisk vs. fysisk objektseparation
- ⚠️ **SAKNAS:** Dynamisk informationsarv
- ⚠️ **SAKNAS:** Artikel-till-objekt fasthakningssystem

Målbild

- 🎯 Fullständig hierarki med både organisatoriska och fysiska nivåer
- 🎯 Tre typer av metadata: fast, fallande, dynamisk
- 🎯 Automatisk orderbyggare som läser hela trädet
- 🎯 Artikelfasthakningssystem med intelligenta matchningar
- 🎯 Orderkoncept som traverserar och skapar uppgifter
- 🎯 Integrerad kundkoppling för avisering och inloggning

4.2 Saknad Funktionalitet

4.2.1 Klusterhierarki

SAKNAS:
- [ ] Utökad hierarkisk modell med fler nivåer:
- Koncern/Organisation
- Juridisk person (BRF, företag)
- Fastighet (befintlig)
- Område/Rum (befintlig)
- Objekt (ny nivå)
- Individuella enheter
- [ ] Tydlig separation mellan organisatoriska och fysiska objekt
- [ ] Flexibel hierarkidjup (inte hårdkodat till 3 nivåer)

4.2.2 Metadatapropagering

SAKNAS:
- [ ] **Fast informationssystem:**
- Metadata som registreras på specifik nivå
- Synlig för undernivåer men ej kopierad
- Referens-baserad åtkomst

- [ ] **Fallande informationssystem:**
- Automatisk ärvning nedåt
- Överskrivningsmekanism (spärr på nivå)
- Prioritetsordning vid konflikt
- [ ] **Dynamisk informationssystem:**
- Uppdaterbar metadata
- Automatisk propagering vid ändring
- Historikspårning (rekommenderat)

4.2.3 Artikel- och Orderkoncept

SAKNAS:
- [ ] **Fasthakningssystem för artiklar:**
- Artikeldefinition med fasthakningstyper
- Matchningslogik objekt ↔ artikel
- Stöd för flera fasthakningar per artikel

- [ ] **Orderkoncept:**
- Skapa orderkoncept separat från ordrar
- Peka orderkoncept mot del av kluster
- Traverseringsalgoritm för att hitta matchande objekt
- Automatisk uppgiftsgenerering
- [ ] **Beroendeartiklar:**
- Definition av beroendeartiklar i struktur
- Automatisk skapande av beroendeuppgifter
- Beroende-kedja hantering

4.2.4 Automatisk Orderbyggare

SAKNAS:
- [ ] **Traverseringsmotor:**
- Läsa klusterhierarki från topp till specifikt objekt
- Samla metadata från varje nivå
- Aggregera till komplett orderunderlag

- [ ] **Orderunderlag generator:**
- Automatisk kompilering av:
  - VAR (plats, tillgång, navigation)
  - VAD (artikel, objekt, antal)
  - TID & RESURS (tider, personal, fordon)
  - FAKTURERING (kund, avtal, pris)
  - KONTAKT (kontaktpersoner på olika nivåer)

4.2.5 Prissystemintegration med Kluster

NULÄGE:
- ✅ Tre-nivå prissystem finns (generell, kund, rabattbrev)
- ⚠️ **SAKNAS:** Koppling till klusterhierarki
- ⚠️ **SAKNAS:** Automatisk prislista-ärvning nedåt

BEHÖVER:
- [ ] Prislista kan kopplas på vilken kluster-nivå som helst
- [ ] Rabattbrev kan kopplas på vilken kluster-nivå som helst
- [ ] Automatisk prioritering vid orderbygge (rabattbrev > kundprislista > generell)
- [ ] Överskrivning av prislista på undernivå

4.2.6 Kundkoppling och Avisering

NULÄGE:
- ⚠️ Kundhantering finns, men koppling till kluster oklar

SAKNAS:
- [ ] **Kundkoppling i klusterhierarki:**
- Koppla kund till organisatorisk nivå
- Fakturamottagare på kundsida
- Flera kontaktpersoner med roller

- [ ] **Aviseringssystem:**
- Automatisk avisering vid orderskapande
- Notifiera kontakter baserat på kluster-nivå
- Olika aviseringstyper (orderbekräftelse, utförande, slutfört)

- [ ] **Kundinloggning:**

- Kunder kan logga in
- Se endast sina objekt i klusterhierarkin
- Roller och behörigheter baserat på kluster-nivå

4.2.7 “Snöret” - Planeringsoptimering

NULÄGE:
- ✅ WeekPlanner med drag-and-drop finns

- ✅ AI Planning Assistant finns
- ✅ AI Auto-Scheduling med väderoptimering finns

SAKNAS:
- [ ] **Tre-vägs tidsoptimering:**
- Objektets/kundens tider (tidsfönster, schema)
- Uppgiftens tider (önskad leverans, deadline, produktionstid)
- Utförarens/teamets tider (tillgänglighet, restid, raster)

- [ ] **Tidskategorier:**
- Egentid (ledighet, vila)
- Förberedelsetid (före produktion)
- Automatisk restid (beräknad)
- Produktionstid (från artikel)
- Beroende-tider
- [ ] **Snöret-visualisering:**
- Visuell representation av “snöret”
- Plan vs. utfall-jämförelse
- Beläggning och veckomål
- [ ] **Nyckeltal för “Snöret”:**
- Procent av veckomål
- Kalkylerad arbetstid totalt
- Producerad tid
- Restid till/från uppdragsort
- Beläggning

4.2.8 Felanmälningar och Avvikelser

SAKNAS:
- [ ] Felanmälningssystem kopplat till kluster
- [ ] Automatisk uppgiftsgenerering från felanmälan
- [ ] Sökfunktion för felanmälningar i kluster

4.3 Databasschema - Identifierade Brister

Nuvarande Schema (Antagande baserat på funktionalitet)

Troliga befintliga tabeller:
- `clusters` (eller `areas`)
- `properties` (fastigheter)
- `rooms` (rum)
- `objects` (allmänna objekt)
- `orders`
- `articles`
- `customers`
- `priceLists`
- `discountLetters`
- `resources`
- `vehicles`

Behövda Schemaändringar

NYA TABELLER:

1. **`cluster_hierarchy`**
   - `id` (UUID, primär)
   - `name` (string)
   - `type` (enum: ‘organization’, ‘legal_entity’, ‘property’, ‘area’, ‘object’, ‘unit’)
   - `parent_id` (UUID, själv-referens, nullable för rotnivå)
   - `level` (integer, djup i hierarkin)
   - `tenant_id` (UUID, för multi-tenancy)
   - Metadata-kolumner se nedan

2. **`cluster_metadata`**
   - `id` (UUID, primär)
   - `cluster_id` (UUID, referens till cluster_hierarchy)
   - `key` (string, t.ex. ‘access_code’, ‘invoice_email’)
   - `value` (jsonb, flexibelt dataformat)
   - `metadata_type` (enum: ‘fixed’, ‘cascading’, ‘dynamic’)
   - `can_override` (boolean, om undernivåer kan överskriva)
   - `created_at` (timestamp)
   - `updated_at` (timestamp)
   - `updated_by` (UUID, användare)

3. **`article_hooks`**
   - `id` (UUID, primär)
   - `article_id` (UUID, referens till articles)
   - `hook_type` (enum: ‘KÄRL’, ‘KÄRLMAT’, ‘KÄRLREST’, ‘RUM’, ‘KOD’, etc.)
   - `priority` (integer, vid flera hooks)

4. **`order_concepts`**
   - `id` (UUID, primär)
   - `name` (string)
   - `description` (text)
   - `target_cluster_id` (UUID, referens till cluster_hierarchy)
   - `target_level` (string, vilken nivå att börja söka från)
   - `desired_delivery_date` (date, nullable)
   - `delivery_period_start` (date, nullable)
   - `delivery_period_end` (date, nullable)
   - `price_list_id` (UUID, referens till priceLists)
   - `customer_id` (UUID, referens till customers)
   - `status` (enum: ‘draft’, ‘active’, ‘completed’)
   - `created_at` (timestamp)

5. **`order_concept_articles`**
   - `id` (UUID, primär)
   - `order_concept_id` (UUID, referens till order_concepts)
   - `article_id` (UUID, referens till articles)
   - `quantity` (integer, nullable om multipliceras)
   - `multiply_by_objects` (boolean)

6. **`task_dependencies`**
   - `id` (UUID, primär)

- `parent_task_id` (UUID, referens till tasks)
- `dependent_task_id` (UUID, referens till tasks)
- `dependency_type` (enum: ‘before’, ‘after’, ‘parallel’)

7. **`cluster_contacts`**
   - `id` (UUID, primär)
   - `cluster_id` (UUID, referens till cluster_hierarchy)
   - `contact_type` (enum: ‘primary’, ‘technical’, ‘billing’, ‘emergency’)
   - `name` (string)
   - `phone` (string)
   - `email` (string)
   - `notify_on_order_create` (boolean)
   - `notify_on_order_complete` (boolean)

8. **`notifications_log`**
   - `id` (UUID, primär)
   - `cluster_id` (UUID, referens till cluster_hierarchy)
   - `contact_id` (UUID, referens till cluster_contacts)
   - `notification_type` (enum: ‘order_confirmation’, ‘work_started’, ‘work_completed’)
   - `order_id` (UUID, referens till orders)
   - `sent_at` (timestamp)
   - `delivery_status` (enum: ‘sent’, ‘delivered’, ‘failed’)

UPPDATERINGAR TILL BEFINTLIGA TABELLER:

1. **`objects` (uppdatera)**
   - Lägg till: `cluster_id` (UUID, referens till cluster_hierarchy)
   - Lägg till: `object_type` (string, för matchning mot article_hooks)
   - Lägg till: `properties` (jsonb, flexibla objektegenskaper)

2. **`articles` (uppdatera)**
   - Lägg till: `has_dependencies` (boolean)
   - Lägg till: `standard_production_time` (integer, minuter)

3. **`orders` (uppdatera)**
   - Lägg till: `order_concept_id` (UUID, referens till order_concepts, nullable)
   - Lägg till: `source_cluster_id` (UUID, vilken klusternivå som genererade ordern)
   - Lägg till: `compiled_metadata` (jsonb, alla ärvda metadata för snabb åtkomst)

4. **`tasks` (ny tabell eller uppdatera `orders` om tasks = orders)**
   - `id` (UUID, primär)
   - `order_id` (UUID, referens till orders)
   - `article_id` (UUID, referens till articles)
   - `object_id` (UUID, referens till objects)
   - `resource_id` (UUID, referens till resources, nullable)
   - `status` (enum: flera statusar enligt planeringsprocessen)
   - `planned_start` (timestamp)
   - `planned_end` (timestamp)
   - `actual_start` (timestamp, nullable)
   - `actual_end` (timestamp, nullable)

    - `production_time_planned` (integer, minuter)
    - `production_time_actual` (integer, minuter, nullable)
    - `travel_time_planned` (integer, minuter)
    - `travel_time_actual` (integer, minuter, nullable)
    - `preparation_time` (integer, minuter)
    - `task_type` (enum: ‘production’, ‘preparation’, ‘travel’, ‘dependent’)
5. **`customers` (uppdatera)**
    - Lägg till: `primary_cluster_id` (UUID, referens till cluster_hierarchy)
6. **`priceLists` (uppdatera)**
    - Lägg till: `cluster_id` (UUID, referens till cluster_hierarchy, nullable)
    - Logik: Om cluster_id är satt, gäller prislistan för den kluster-nivån och nedåt
7. **`discountLetters` (uppdatera)**
    - Lägg till: `cluster_id` (UUID, referens till cluster_hierarchy, nullable)

4.4 API-endpoints som Saknas

Klusterhantering

- [ ] `POST /api/clusters` - Skapa nytt kluster
- [ ] `GET /api/clusters/:id/hierarchy` - Hämta komplett hierarki
- [ ] `GET /api/clusters/:id/ancestors` - Hämta alla föräldrar uppåt
- [ ] `GET /api/clusters/:id/descendants` - Hämta alla barn nedåt
- [ ] `POST /api/clusters/:id/metadata` - Lägg till metadata
- [ ] `PUT /api/clusters/:id/metadata/:key` - Uppdatera metadata
- [ ] `GET /api/clusters/:id/compiled-metadata` - Hämta all ärvd metadata

Orderkoncept

- [ ] `POST /api/order-concepts` - Skapa orderkoncept
- [ ] `POST /api/order-concepts/:id/execute` - Exekvera orderkoncept (generera uppgifter)
- [ ] `GET /api/order-concepts/:id/preview` - Förhandsgranska matchade objekt
- [ ] `GET /api/order-concepts/:id/generated-tasks` - Hämta genererade uppgifter

Artikelfasthakningar

- [ ] `POST /api/articles/:id/hooks` - Lägg till fasthakningar
- [ ] `GET /api/articles/by-hook/:hookType` - Hämta artiklar med specifik hook

Orderbyggare

- [ ] `POST /api/orders/build` - Bygg komplett order från kluster och artikel
- [ ] `GET /api/orders/:id/source-metadata` - Hämta metadata-källor för order

Avisering

- [ ] `POST /api/notifications/send` - Skicka avisering
- [ ] `GET /api/clusters/:id/contacts` - Hämta kontakter för kluster-nivå
- [ ] `POST /api/clusters/:id/contacts` - Lägg till kontakt

“Snöret” - Planering

- [ ] `GET /api/planning/string/:resourceId/:week` - Hämta “snöret” för resurs och vecka
- [ ] `POST /api/planning/optimize-string` - Optimera “snöret” (tre-vägs optimering)

- [ ] `GET /api/planning/metrics/:resourceId/:week` - Hämta nyckeltal för “snöret”

4.5 Prioriterade Utvecklingsområden

🔴 KRITISK PRIORITET (Måste implementeras först)

1. **Klusterhierarki med Metadatapropagering**
   - Databasschemat för `cluster_hierarchy` och `cluster_metadata`
   - API för att skapa och hantera kluster
   - Traverseringslogik för att hämta ärvd metadata
   - Överskrivningsmekanism

2. **Artikel-till-Objekt Fasthakningssystem**
   - Databasschemat för `article_hooks`
   - Matchningslogik
   - API för fasthakningshantering

3. **Orderkoncept med Automatisk Uppgiftsgenerering**
   - Databasschemat för `order_concepts` och `order_concept_articles`
   - Traverseringsalgoritm
   - Uppgiftsgenerering baserat på matchningar
   - API för orderkoncept

🟠 HÖG PRIORITET (Behövs för fullständig funktionalitet)

1. **Automatisk Orderbyggare**
   - Metadata-aggregering från klusterhierarki
   - Komplett orderunderlag generator
   - API för orderbygge

2. **Beroendeartiklar och Uppgiftskedjor**
   - Databasschemat för `task_dependencies`
   - Automatisk skapande av beroendeuppgifter
   - Beroende-logik i orderkonceptexekvering

3. **Kundkoppling och Aviseringssystem**
   - Koppla kunder till kluster
   - Kontaktpersoner på klusternivå
   - Automatisk avisering vid orderhändelser
   - Kundinloggning med kluster-baserad åtkomst

🟡 MEDEL PRIORITET (Förbättrar funktionalitet)

1. **“Snöret” - Tre-vägs Planeringsoptimering**
   - Implementera tidskategorier
   - Optimeringsalgoritm för tre tidsintressen
   - “Snöret”-visualisering i UI
   - Nyckeltal och rapporter

2. **Prissystemintegration med Kluster**
   - Koppla prislistor och rabattbrev till kluster-nivåer
   - Automatisk prislista-ärvning
   - Prioritering vid orderbygge

3. **Felanmälningar Kopplade till Kluster**
   - Felanmälningssystem

- Automatisk uppgiftsgenerering från felanmälan
- Sökfunktion i kluster

🟢 LÅG PRIORITET (Nice-to-have)

1. **Avancerad Kluster-visualisering**
   - Grafisk trädvy av kluster
   - Interaktiv navigering
   - Visuell representation av metadata-flöde
2. **Historikspårning för Dynamisk Metadata**
   - Spara ändringshistorik
   - Vem ändrade vad och när
   - Återställningsfunktion
3. **Avancerad Analys av "Snöret"**
   - Historiska trender
   - Jämförelser mellan resurser
   - Prediktiv analys för beläggning

---

5. Implementationsguide för Replit 3.0

5.1 Övergripande Strategi

Implementationen ska ske **iterativt** med fokus på att först bygga kärnfunktionalitet (klusterhierarki och metadatapropagering), sedan lägga till intelligens (orderkoncept och automatisk orderbyggare), och slutligen integrera med befintliga system (prissystem, avisering, planering).

Fas 1: Klusterhierarki och Metadatapropagering (Kritisk)
Fas 2: Artikelfasthakningar och Orderkoncept (Kritisk)
Fas 3: Automatisk Orderbyggare (Hög Prioritet)
Fas 4: Integration och Förbättringar (Medel-Låg Prioritet)

5.2 Fas 1: Klusterhierarki och Metadatapropagering

Steg 1.1: Databasschema för Klusterhierarki

Fil: `db/schema/clusterHierarchy.ts` (ny fil)

Skapa tabellen `cluster_hierarchy`:

```
import { pgTable, uuid, varchar, integer, timestamp, pgEnum } from 'drizzle-orm/pg-core';

export const clusterTypeEnum = pgEnum('cluster_type', [
  'organization',
  'legal_entity',
  'property',
  'area',
  'room',
  'object',
  'unit'
]);

export const clusterHierarchy = pgTable('cluster_hierarchy', {
  id: uuid('id').primaryKey().defaultRandom(),
  name: varchar('name', { length: 255 }).notNull(),
  type: clusterTypeEnum('type').notNull(),
  parentId: uuid('parent_id').references(() => clusterHierarchy.id),
  level: integer('level').notNull(), // 0 = root, 1 = child of root, etc.
  tenantId: uuid('tenant_id').notNull(), // För multi-tenancy
  createdAt: timestamp('created_at').defaultNow(),
  updatedAt: timestamp('updated_at').defaultNow(),
});

// Index för snabb traversering
// CREATE INDEX idx_cluster_parent ON cluster_hierarchy(parent_id);
// CREATE INDEX idx_cluster_tenant ON cluster_hierarchy(tenant_id);
// CREATE INDEX idx_cluster_type ON cluster_hierarchy(type);
```

Steg 1.2: Databasschema för Klustermetadata

Fil: `db/schema/clusterMetadata.ts` (ny fil)

```
import { pgTable, uuid, varchar, jsonb, timestamp, boolean, pgEnum } from 'drizzle-orm/pg-core';
import { clusterHierarchy } from './clusterHierarchy';

export const metadataTypeEnum = pgEnum('metadata_type', [
  'fixed',      // Fast information
  'cascading',  // Fallande information
  'dynamic'     // Dynamisk information
]);

export const clusterMetadata = pgTable('cluster_metadata', {
  id: uuid('id').primaryKey().defaultRandom(),
  clusterId: uuid('cluster_id').references(() => clusterHierarchy.id).notNull(),
  key: varchar('key', { length: 100 }).notNull(),
  value: jsonb('value').notNull(), // Flexibelt format
  metadataType: metadataTypeEnum('metadata_type').notNull(),
  canOverride: boolean('can_override').default(true), // Om undernivåer kan överskriva
  createdAt: timestamp('created_at').defaultNow(),
  updatedAt: timestamp('updated_at').defaultNow(),
  updatedBy: uuid('updated_by'), // Referens till users table
});

// Composite index för snabb sökning av metadata
// CREATE INDEX idx_cluster_metadata_cluster_key ON cluster_metadata(cluster_id, key);
```

Vanliga metadata-nycklar (exempel):

- `address` - Adress

- `coordinates` - { lat, lng }
- `access_code` - Nyckelkod
- `invoice_email` - Fakturamejl
- `contract_type` - Avtalstyp
- `price_list_id` - Prislista ID
- `photos` - Array av foto-URL:er
- `availability_hours` - Tillgänglighetstider
- `contact_instructions` - Särskilda instruktioner

Steg 1.3: Migrering av Befintliga Objekt till Klusterhierarki

Strategi: Behåll befintliga tabeller (`areas` , `properties` , `rooms` , `objects`) men skapa en mappning till `cluster_hierarchy` .

Alternativ 1 - Gradvis Migrering:
- Lägg till kolumn `cluster_id` i befintliga tabeller
- Skapa klusterposter för varje befintligt objekt
- Länka via `cluster_id`

Alternativ 2 - Komplett Omstrukturering:
- Ersätt befintliga separata tabeller med enhetlig `cluster_hierarchy`
- Migrera all data
- Uppdatera alla referenser

Rekommendation: Alternativ 1 för att minimera breaking changes.

Migreringsskript: `db/migrations/001_add_cluster_hierarchy.ts`

```
// Pseudo-kod för migrering
async function migrate() {
  // 1. Skapa cluster_hierarchy tabell
  await createClusterHierarchyTable();

  // 2. Migrera befintliga areas till cluster_hierarchy (type='area', level=0)
  const areas = await db.select().from(areasTable);
  for (const area of areas) {
    const clusterId = await db.insert(clusterHierarchy).values({
      name: area.name,
      type: 'area',
      parentId: null,
      level: 0,
      tenantId: area.tenantId,
    }).returning({ id: clusterHierarchy.id });

    // Uppdatera areas tabell med cluster_id
    await db.update(areasTable)
      .set({ clusterId: clusterId.id })
      .where(eq(areasTable.id, area.id));
  }

  // 3. Migrera properties (type='property', level=1, parent=area's cluster_id)
  const properties = await db.select().from(propertiesTable);
  for (const property of properties) {
    const parentArea = await db.select()
      .from(areasTable)
      .where(eq(areasTable.id, property.areaId))
      .limit(1);

    const clusterId = await db.insert(clusterHierarchy).values({
      name: property.name,
      type: 'property',
      parentId: parentArea[0].clusterId,
      level: 1,
      tenantId: property.tenantId,
    }).returning({ id: clusterHierarchy.id });

    await db.update(propertiesTable)
      .set({ clusterId: clusterId.id })
      .where(eq(propertiesTable.id, property.id));
  }

  // 4. Migrera rooms (type='room', level=2)
  // ... liknande logik

  // 5. Migrera befintlig metadata till cluster_metadata
  // ... samla adresser, koordinater, etc. och skapa cluster_metadata poster
}
```

Steg 1.4: API för Klusterhantering

Fil: `server/routes/clusters.ts` (ny fil)

Endpoints att implementera:

```
import express from 'express';
const router = express.Router();

// Skapa nytt kluster
router.post('/', async (req, res) => {
  const { name, type, parentId, tenantId } = req.body;

  // Validering
  // Beräkna level baserat på parent
  // Skapa kluster
  // Returnera kluster
});

// Hämta kluster med komplett hierarki (alla föräldrar och barn)
router.get('/:id/hierarchy', async (req, res) => {
  const { id } = req.params;

  // Implementera rekursiv query för att hämta hela trädet
  // Returnera hierarkisk struktur
});

// Hämta alla föräldrar uppåt (ancestors)
router.get('/:id/ancestors', async (req, res) => {
  const { id } = req.params;

  // Recursive CTE (Common Table Expression) för att hämta föräldrar
  // SQL exempel:
  // WITH RECURSIVE ancestors AS (
  //   SELECT * FROM cluster_hierarchy WHERE id = $1
  //   UNION ALL
  //   SELECT ch.* FROM cluster_hierarchy ch
  //   INNER JOIN ancestors a ON ch.id = a.parent_id
  // )
  // SELECT * FROM ancestors;
});

// Hämta alla barn nedåt (descendants)
router.get('/:id/descendants', async (req, res) => {
  const { id } = req.params;

  // Recursive CTE för att hämta barn
});

// Lägg till metadata till kluster
router.post('/:id/metadata', async (req, res) => {
  const { id } = req.params;
  const { key, value, metadataType, canOverride } = req.body;

  // Skapa cluster_metadata post
});

// Uppdatera metadata (endast dynamic metadata bör kunna uppdateras)
router.put('/:id/metadata/:key', async (req, res) => {
  const { id, key } = req.params;
  const { value } = req.body;

  // Uppdatera cluster_metadata
  // Uppdatera updatedAt och updatedBy
});

// Hämta all ärvd metadata för ett kluster (kompilerad vy)
router.get('/:id/compiled-metadata', async (req, res) => {
```

```
  const { id } = req.params;

  // Implementera metadata-aggregering:
  // 1. Hämta alla ancestors
  // 2. För varje ancestor, hämta metadata
  // 3. Börja från root och gå nedåt
  // 4. För varje metadata:
  //    - Om fixed: inkludera som referens
  //    - Om cascading: inkludera värde, men tillåt överskrivning
  //    - Om dynamic: inkludera värde, tillåt överskrivning
  // 5. Returnera kompilerad metadata-objekt
});

export default router;
```

Steg 1.5: Service-lager för Metadatapropagering

Fil: `server/services/clusterMetadataService.ts` (ny fil)

Kärnfunktioner:

```
import { db } from '../db';
import { clusterHierarchy, clusterMetadata } from '../db/schema';

export class ClusterMetadataService {

  /**
   * Hämta alla ancestors för ett kluster
   */
  async getAncestors(clusterId: string): Promise<ClusterHierarchy[]> {
    // Recursive CTE query
    const result = await db.execute(sql`
      WITH RECURSIVE ancestors AS (
        SELECT * FROM cluster_hierarchy WHERE id = ${clusterId}
        UNION ALL
        SELECT ch.*
        FROM cluster_hierarchy ch
        INNER JOIN ancestors a ON ch.id = a.parent_id
      )
      SELECT * FROM ancestors ORDER BY level ASC
    `);

    return result.rows;
  }

  /**
   * Hämta alla descendants för ett kluster
   */
  async getDescendants(clusterId: string): Promise<ClusterHierarchy[]> {
    // Recursive CTE query
    const result = await db.execute(sql`
      WITH RECURSIVE descendants AS (
        SELECT * FROM cluster_hierarchy WHERE id = ${clusterId}
        UNION ALL
        SELECT ch.*
        FROM cluster_hierarchy ch
        INNER JOIN descendants d ON ch.parent_id = d.id
      )
      SELECT * FROM descendants ORDER BY level ASC
    `);

    return result.rows;
  }

  /**
   * Kompilera all ärvd metadata för ett kluster
   * Detta är kärnfunktionen för "fallande information"
   */
  async compileMetadata(clusterId: string): Promise<Record<string, any>> {
    // 1. Hämta alla ancestors (från root till aktuellt kluster)
    const ancestors = await this.getAncestors(clusterId);

    // 2. Sortera från root till leaf
    ancestors.sort((a, b) => a.level - b.level);

    // 3. Aggregera metadata
    const compiledMetadata: Record<string, any> = {};

    for (const ancestor of ancestors) {
      // Hämta all metadata för denna nivå
      const metadata = await db.select()
        .from(clusterMetadata)
        .where(eq(clusterMetadata.clusterId, ancestor.id));
```

```
      for (const meta of metadata) {
        const { key, value, metadataType, canOverride } = meta;

        if (metadataType === 'fixed') {
          // Fast information: Lägg till som referens om inte redan finns
          if (!compiledMetadata[key]) {
            compiledMetadata[key] = {
              value,
              source: ancestor.id,
              type: 'fixed',
              level: ancestor.level,
            };
          }
        } else if (metadataType === 'cascading' || metadataType === 'dynamic') {
          // Fallande/Dynamisk information: Överskriver tidigare värde om tillåtet
          if (!compiledMetadata[key] || canOverride) {
            compiledMetadata[key] = {
              value,
              source: ancestor.id,
              type: metadataType,
              level: ancestor.level,
            };
          }
        }
      }
    }

    return compiledMetadata;
  }

  /**
   * Propagera uppdatering av dynamisk metadata till alla descendants
   * (För framtida optimering: cache invalidation)
   */
  async propagateMetadataUpdate(clusterId: string, key: string, newValue: any) {
    // Uppdatera själva metadata
    await db.update(clusterMetadata)
      .set({ value: newValue, updatedAt: new Date() })
      .where(
        and(
          eq(clusterMetadata.clusterId, clusterId),
          eq(clusterMetadata.key, key)
        )
      );

    // Invalidera cache för alla descendants (om caching används)
    const descendants = await this.getDescendants(clusterId);
    for (const descendant of descendants) {
      // cache.invalidate(`cluster_metadata:${descendant.id}`);
    }
  }
}
```

Steg 1.6: Frontend-komponenter för Klusterhierarki

Fil: `client/src/components/ClusterTreeView.tsx` (ny komponent)

Funktionalitet:
- Visuell trädvy av klusterhierarki
- Expandera/kollapsera noder

- Visa metadata för varje nivå
- Indikera var metadata kommer ifrån (ärv vs. egen)

Skiss:

```
import React, { useState, useEffect } from 'react';
import { ChevronRight, ChevronDown } from 'lucide-react';

interface ClusterNode {
  id: string;
  name: string;
  type: string;
  level: number;
  children?: ClusterNode[];
  metadata?: Record<string, any>;
}

export const ClusterTreeView: React.FC<{ rootClusterId: string }> = ({ rootClusterId }
) => {
  const [tree, setTree] = useState<ClusterNode | null>(null);
  const [expandedNodes, setExpandedNodes] = useState<Set<string>>(new Set());

  useEffect(() => {
    // Hämta hierarki från API
    fetch(`/api/clusters/${rootClusterId}/hierarchy`)
      .then(res => res.json())
      .then(data => setTree(data));
  }, [rootClusterId]);

  const toggleNode = (nodeId: string) => {
    setExpandedNodes(prev => {
      const newSet = new Set(prev);
      if (newSet.has(nodeId)) {
        newSet.delete(nodeId);
      } else {
        newSet.add(nodeId);
      }
      return newSet;
    });
  };

  const renderNode = (node: ClusterNode) => {
    const isExpanded = expandedNodes.has(node.id);
    const hasChildren = node.children && node.children.length > 0;

    return (
      <div key={node.id} style={{ marginLeft: node.level * 20 }}>
        <div
          onClick={() => toggleNode(node.id)}
          className="flex items-center cursor-pointer hover:bg-gray-100 p-2"
        >
          {hasChildren && (
            isExpanded ? <ChevronDown /> : <ChevronRight />
          )}
          <span className="font-semibold">{node.name}</span>
          <span className="text-gray-500 ml-2">({node.type})</span>
        </div>

        {/* Visa metadata för noden */}
        {isExpanded && node.metadata && (
          <div className="ml-8 text-sm text-gray-600">
            {Object.entries(node.metadata).map(([key, meta]) => (
              <div key={key}>
                <strong>{key}:</strong> {JSON.stringify(meta.value)}
                <span className="text-xs text-gray-400 ml-2">
                  ({meta.type}, från nivå {meta.level})
                </span>
```

```
                </div>
              ))}
            </div>
          )}

          {/* Rekursivt rendera barn */}
          {isExpanded && hasChildren && node.children!.map(child => renderNode(child))}
        </div>
      );
    };

    if (!tree) return <div>Laddar...</div>;

    return (
      <div className="cluster-tree-view">
        <h2>Klusterhierarki</h2>
        {renderNode(tree)}
      </div>
    );
  };
```

Steg 1.7: Tester för Metadatapropagering

Fil: `server/tests/clusterMetadata.test.ts`

Testfall:

```
import { describe, it, expect } from 'vitest';
import { ClusterMetadataService } from '../services/clusterMetadataService';

describe('ClusterMetadataService', () => {
  const service = new ClusterMetadataService();

  it('should inherit cascading metadata from parent', async () => {
    // Skapa test-hierarki
    // Castor (root) -> BRF Huset -> Fastighet Grytan 5

    // Castor har metadata: invoice_email = "ekonomi@castor.se" (cascading)
    // BRF Huset har ingen egen invoice_email
    // Fastighet Grytan 5 har ingen egen invoice_email

    const compiled = await service.compileMetadata('fastighet_grytan_5_id');

    expect(compiled.invoice_email.value).toBe('ekonomi@castor.se');
    expect(compiled.invoice_email.source).toBe('castor_id');
  });

  it('should allow override of cascading metadata', async () => {
    // Castor har metadata: invoice_email = "ekonomi@castor.se" (cascading, canOver-
ride=true)
    // BRF Huset har metadata: invoice_email = "faktura@brfhuset.se" (cascading)
    // Fastighet Grytan 5 har ingen egen invoice_email

    const compiled = await service.compileMetadata('fastighet_grytan_5_id');

    expect(compiled.invoice_email.value).toBe('faktura@brfhuset.se');
    expect(compiled.invoice_email.source).toBe('brf_huset_id');
  });

  it('should keep fixed metadata at source level', async () => {
    // Fastighet Grytan 5 har metadata: access_code = "100" (fixed)

    const compiled = await service.compileMetadata('rum_id_under_grytan_5');

    expect(compiled.access_code.value).toBe('100');
    expect(compiled.access_code.type).toBe('fixed');
    expect(compiled.access_code.source).toBe('fastighet_grytan_5_id');
  });

  it('should propagate dynamic metadata update to descendants', async () => {
    // Castor uppdaterar invoice_email från "ekonomi@castor.se" till "fak-
tura@castor.se"

    await service.propagateMetadataUpdate('castor_id', 'invoice_email', 'fak-
tura@castor.se');

    // Alla descendants bör se den nya emailen
    const compiled = await service.compileMetadata('fastighet_grytan_5_id');
    expect(compiled.invoice_email.value).toBe('faktura@castor.se');
  });
});
```

5.3 Fas 2: Artikelfasthakningar och Orderkoncept

Steg 2.1: Databasschema för Artikelfasthakningar

Fil: `db/schema/articleHooks.ts` (ny fil)

```
import { pgTable, uuid, varchar, integer, pgEnum } from 'drizzle-orm/pg-core';
import { articles } from './articles';

// Definiera vanliga hook-typer
// (Detta kan utökas dynamiskt i framtiden)
export const hookTypeEnum = pgEnum('hook_type', [
  'KÄRL',           // Alla kärl
  'KÄRLMAT',        // Matavfallskärl
  'KÄRLREST',       // Restavfallskärl
  'KÄRLPAPPER',     // Papperskärl
  'RUM',            // Alla rum
  'ÅTERVINNINGSRUM', // Återvinningsrum
  'KOD',            // Objekt med nyckelkod
  'FASTIGHET',      // Fastigheter
  'OMRÅDE',         // Områden
]);

export const articleHooks = pgTable('article_hooks', {
  id: uuid('id').primaryKey().defaultRandom(),
  articleId: uuid('article_id').references(() => articles.id).notNull(),
  hookType: hookTypeEnum('hook_type').notNull(),
  priority: integer('priority').default(0), // Vid flera hooks, högre priority först
});

// Index för snabb sökning
// CREATE INDEX idx_article_hooks_article ON article_hooks(article_id);
// CREATE INDEX idx_article_hooks_type ON article_hooks(hook_type);
```

Steg 2.2: Uppdatera Articles-schema

Fil: `db/schema/articles.ts` (uppdatera befintlig)

```
// Lägg till i befintlig articles-tabell
export const articles = pgTable('articles', {
  // ... befintliga kolumner

  hasDependencies: boolean('has_dependencies').default(false),
  standardProductionTime: integer('standard_production_time'), // minuter

  // Om artikel är en beroendeprodukt
  isDependency: boolean('is_dependency').default(false),
  parentArticleId: uuid('parent_article_id').references(() => articles.id),
});
```

Steg 2.3: Uppdatera Objects-schema för Matchning

Fil: `db/schema/objects.ts` (uppdatera befintlig)

```
// Lägg till i befintlig objects-tabell
export const objects = pgTable('objects', {
  // ... befintliga kolumner

  clusterId: uuid('cluster_id').references(() => clusterHierarchy.id),
  objectType: varchar('object_type', { length: 50 }), // t.ex. 'KÄRLMAT', 'RUM'
  properties: jsonb('properties'), // Flexibla objektegenskaper
});

// För befintliga objekt utan objectType, skapa migrering:
// UPDATE objects SET object_type = 'KÄRL' WHERE type = 'bin';
// UPDATE objects SET object_type = 'KÄRLMAT' WHERE type = 'bin' AND category =
'food_waste';
```

Steg 2.4: Databasschema för Orderkoncept

Fil: `db/schema/orderConcepts.ts` (ny fil)

```
import { pgTable, uuid, varchar, text, date, timestamp, pgEnum } from 'drizzle-orm/pg-
core';
import { clusterHierarchy } from './clusterHierarchy';
import { customers } from './customers';
import { priceLists } from './priceLists';

export const orderConceptStatusEnum = pgEnum('order_concept_status', [
  'draft',
  'active',
  'completed',
  'cancelled'
]);

export const orderConcepts = pgTable('order_concepts', {
  id: uuid('id').primaryKey().defaultRandom(),
  name: varchar('name', { length: 255 }).notNull(),
  description: text('description'),

  // Målinriktning i klustret
  targetClusterId: uuid('target_cluster_id').references(() => clusterHier-
archy.id).notNull(),
  targetLevel: varchar('target_level', { length: 50 }), // t.ex. 'property', 'room',
'all'

  // Tidsinformation
  desiredDeliveryDate: date('desired_delivery_date'),
  deliveryPeriodStart: date('delivery_period_start'),
  deliveryPeriodEnd: date('delivery_period_end'),

  // Affärsinformation
  priceListId: uuid('price_list_id').references(() => priceLists.id),
  customerId: uuid('customer_id').references(() => customers.id),

  status: orderConceptStatusEnum('status').default('draft'),

  createdAt: timestamp('created_at').defaultNow(),
  updatedAt: timestamp('updated_at').defaultNow(),
  executedAt: timestamp('executed_at'), // När orderkonceptet exekverades
});
```

Fil: `db/schema/orderConceptArticles.ts` (ny fil)

```
import { pgTable, uuid, integer, boolean } from 'drizzle-orm/pg-core';
import { orderConcepts } from './orderConcepts';
import { articles } from './articles';

export const orderConceptArticles = pgTable('order_concept_articles', {
  id: uuid('id').primaryKey().defaultRandom(),
  orderConceptId: uuid('order_concept_id').references(() => orderCon-
cepts.id).notNull(),
  articleId: uuid('article_id').references(() => articles.id).notNull(),
  quantity: integer('quantity'), // null om multipliceras per objekt
  multiplyByObjects: boolean('multiply_by_objects').default(true),
});

// Index
// CREATE INDEX idx_order_concept_articles_concept ON or-
der_concept_articles(order_concept_id);
```

Steg 2.5: Service för Orderkoncept-exekvering

Fil: `server/services/orderConceptService.ts` (ny fil)

Kärnlogik:

```
import { db } from '../db';
import { orderConcepts, orderConceptArticles, articleHooks, objects, orders } from '..
/db/schema';
import { ClusterMetadataService } from './clusterMetadataService';

export class OrderConceptService {
  private metadataService = new ClusterMetadataService();

  /**
   * Exekvera ett orderkoncept:
   * 1. Hämta alla artiklar i orderkonceptet
   * 2. För varje artikel, hitta matchande objekt i klustret
   * 3. För varje matchning, skapa en uppgift/order
   * 4. Hantera beroendeartiklar
   * 5. Skicka aviseringar
   */
  async executeOrderConcept(orderConceptId: string): Promise<ExecutionResult> {
    // 1. Hämta orderkoncept
    const orderConcept = await db.select()
      .from(orderConcepts)
      .where(eq(orderConcepts.id, orderConceptId))
      .limit(1);

    if (!orderConcept.length) {
      throw new Error('Order concept not found');
    }

    const concept = orderConcept[0];

    // 2. Hämta alla artiklar i orderkonceptet
    const conceptArticles = await db.select()
      .from(orderConceptArticles)
      .leftJoin(articles, eq(orderConceptArticles.articleId, articles.id))
      .where(eq(orderConceptArticles.orderConceptId, orderConceptId));

    const createdTasks: any[] = [];

    // 3. För varje artikel
    for (const conceptArticle of conceptArticles) {
      const article = conceptArticle.articles;

      // Hitta matchande objekt
      const matchingObjects = await this.findMatchingObjects(
        concept.targetClusterId,
        article.id
      );

      // 4. För varje matchande objekt, skapa uppgift
      for (const obj of matchingObjects) {
        const task = await this.createTask(
          orderConceptId,
          article,
          obj,
          concept
        );
        createdTasks.push(task);

        // 5. Om artikel har beroendeartiklar, skapa dem
        if (article.hasDependencies) {
          const dependencyTasks = await this.createDependencyTasks(
            task.id,
            article.id,
```

```
            obj
          );
          createdTasks.push(...dependencyTasks);
        }
      }
    }

    // 6. Uppdatera orderkoncept status
    await db.update(orderConcepts)
      .set({
        status: 'active',
        executedAt: new Date(),
        updatedAt: new Date()
      })
      .where(eq(orderConcepts.id, orderConceptId));

    // 7. Skicka aviseringar (implementeras i Fas 4)
    // await this.sendNotifications(createdTasks);

    return {
      orderConceptId,
      tasksCreated: createdTasks.length,
      tasks: createdTasks,
    };
  }

  /**
   * Hitta alla objekt som matchar artikelns fasthakningar
   * inom målklustret
   */
  async findMatchingObjects(targetClusterId: string, articleId: string):
Promise<any[]> {
    // 1. Hämta artikelns fasthakningar
    const hooks = await db.select()
      .from(articleHooks)
      .where(eq(articleHooks.articleId, articleId));

    if (!hooks.length) {
      return []; // Ingen hook = ingen matchning
    }

    // 2. Hämta alla descendants av målklustret
    const descendants = await this.metadataService.getDescendants(targetClusterId);
    const clusterIds = descendants.map(d => d.id);

    // 3. Hitta objekt som:
    //    - Finns i någon av descendant-klustren
    //    - Har objectType som matchar någon av hooks
    const hookTypes = hooks.map(h => h.hookType);

    const matchingObjects = await db.select()
      .from(objects)
      .where(
        and(
          inArray(objects.clusterId, clusterIds),
          inArray(objects.objectType, hookTypes)
        )
      );

    return matchingObjects;
  }

  /**
```

```
   * Skapa en uppgift för ett specifikt objekt och artikel
   */
  async createTask(
    orderConceptId: string,
    article: any,
    object: any,
    concept: any
  ): Promise<any> {
    // Hämta kompilerad metadata för objektets kluster
    const metadata = await this.metadataService.compileMetadata(object.clusterId);

    // Hämta prisinformation (från prislista, rabattbrev)
    const pricing = await this.getPricing(article.id, concept.priceListId, concept.cus
tomerId);

    // Skapa order/task
    const task = await db.insert(orders).values({
      orderConceptId,
      articleId: article.id,
      objectId: object.id,
      sourceClusterId: object.clusterId,
      customerId: concept.customerId,

      // Tidsinformation
      desiredDeliveryDate: concept.desiredDeliveryDate,
      plannedStart: null, // Sätts i planeringsprocessen

      // Prisinformation
      salesPrice: pricing.salesPrice,
      cost: pricing.cost,
      productionTime: pricing.productionTime,

      // Kompilerad metadata (för snabb åtkomst)
      compiledMetadata: metadata,

      status: 'pending',
      createdAt: new Date(),
    }).returning();

    return task[0];
  }

  /**
   * Skapa beroendeuppgifter för en huvuduppgift
   */
  async createDependencyTasks(parentTaskId: string, parentArticleId: string, object: a
ny): Promise<any[]> {
    // Hitta beroendeartiklar
    const dependencyArticles = await db.select()
      .from(articles)
      .where(
        and(
          eq(articles.isDependency, true),
          eq(articles.parentArticleId, parentArticleId)
        )
      );

    const dependencyTasks = [];

    for (const depArticle of dependencyArticles) {
      // Skapa task för beroendeartikeln
      const depTask = await db.insert(orders).values({
        articleId: depArticle.id,
```

```
        objectId: object.id,
        parentTaskId: parentTaskId,
        status: 'dependent', // Väntar på huvuduppgift
        // ... andra fält
      }).returning();

      dependencyTasks.push(depTask[0]);
    }

    return dependencyTasks;
  }

  /**
   * Hämta prisinformation med prioritet:
   * 1. Rabattbrev (om tillämpligt)
   * 2. Kundspecifik prislista
   * 3. Generell prislista
   */
  async getPricing(articleId: string, priceListId: string, customerId: string) {
    // Implementera prissökningslogik
    // (detaljer beror på befintligt prissystem)

    return {
      salesPrice: 100,
      cost: 50,
      productionTime: 15, // minuter
    };
  }

  /**
   * Förhandsgranska vilka objekt som skulle matchas
   * (utan att skapa uppgifter)
   */
  async previewOrderConcept(orderConceptId: string): Promise<PreviewResult> {
    const orderConcept = await db.select()
      .from(orderConcepts)
      .where(eq(orderConcepts.id, orderConceptId))
      .limit(1);

    const concept = orderConcept[0];

    const conceptArticles = await db.select()
      .from(orderConceptArticles)
      .leftJoin(articles, eq(orderConceptArticles.articleId, articles.id))
      .where(eq(orderConceptArticles.orderConceptId, orderConceptId));

    const preview: PreviewResult = {
      orderConceptId,
      articles: [],
    };

    for (const conceptArticle of conceptArticles) {
      const article = conceptArticle.articles;
      const matchingObjects = await this.findMatchingObjects(
        concept.targetClusterId,
        article.id
      );

      preview.articles.push({
        articleId: article.id,
        articleName: article.name,
        matchedObjects: matchingObjects.length,
        objects: matchingObjects,
```

```
      });
    }

    return preview;
  }
}

interface ExecutionResult {
  orderConceptId: string;
  tasksCreated: number;
  tasks: any[];
}

interface PreviewResult {
  orderConceptId: string;
  articles: {
    articleId: string;
    articleName: string;
    matchedObjects: number;
    objects: any[];
  }[];
}
```

Steg 2.6: API för Orderkoncept

Fil: `server/routes/orderConcepts.ts` (ny fil)

```
import express from 'express';
import { OrderConceptService } from '../services/orderConceptService';

const router = express.Router();
const service = new OrderConceptService();

// Skapa orderkoncept
router.post('/', async (req, res) => {
  const { name, description, targetClusterId, articles, customerId, priceListId, de-
siredDeliveryDate } = req.body;

  // Validering
  // Skapa order concept
  // Koppla artiklar
  // Returnera
});

// Förhandsgranska orderkoncept
router.get('/:id/preview', async (req, res) => {
  const { id } = req.params;

  try {
    const preview = await service.previewOrderConcept(id);
    res.json(preview);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

// Exekvera orderkoncept (skapa uppgifter)
router.post('/:id/execute', async (req, res) => {
  const { id } = req.params;

  try {
    const result = await service.executeOrderConcept(id);
    res.json(result);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

// Hämta genererade uppgifter för ett orderkoncept
router.get('/:id/tasks', async (req, res) => {
  const { id } = req.params;

  // Hämta alla tasks med orderConceptId = id
  const tasks = await db.select()
    .from(orders)
    .where(eq(orders.orderConceptId, id));

  res.json(tasks);
});

export default router;
```

Steg 2.7: Frontend för Orderkoncept

Fil: `client/src/pages/OrderConceptCreate.tsx` (ny sida)

Funktionalitet:

1. Välj målkluster (dropdown eller trädvy)

2. Lägg till artiklar (sök och välj)
3. Sätt leveranstid/period
4. Välj kund och prislista
5. Förhandsgranska matchningar
6. Exekvera och skapa uppgifter

Skiss:

```
import React, { useState } from 'react';
import { Button } from '@/components/ui/button';
import { ClusterSelector } from '@/components/ClusterSelector';
import { ArticleSelector } from '@/components/ArticleSelector';

export const OrderConceptCreate: React.FC = () => {
  const [targetCluster, setTargetCluster] = useState(null);
  const [selectedArticles, setSelectedArticles] = useState([]);
  const [preview, setPreview] = useState(null);

  const handlePreview = async () => {
    // Skapa tillfälligt orderkoncept
    const conceptId = await createDraftConcept();

    // Hämta förhandsgranskning
    const result = await fetch(`/api/order-concepts/${conceptId}/preview`);
    const data = await result.json();
    setPreview(data);
  };

  const handleExecute = async () => {
    const conceptId = await createDraftConcept();
    await fetch(`/api/order-concepts/${conceptId}/execute`, { method: 'POST' });

    // Redirect till uppgiftslista
  };

  return (
    <div className="p-6">
      <h1 className="text-2xl font-bold mb-4">Skapa Orderkoncept</h1>

      <div className="space-y-4">
        <div>
          <label>Målkluster</label>
          <ClusterSelector
            value={targetCluster}
            onChange={setTargetCluster}
          />
        </div>

        <div>
          <label>Artiklar</label>
          <ArticleSelector
            value={selectedArticles}
            onChange={setSelectedArticles}
          />
        </div>

        <div className="flex gap-2">
          <Button onClick={handlePreview}>Förhandsgranska</Button>
          <Button onClick={handleExecute} variant="primary">Skapa Uppgifter</Button>
        </div>

        {preview && (
          <div className="mt-4 border p-4">
            <h2 className="text-xl font-semibold">Förhandsgranskning</h2>
            {preview.articles.map(article => (
              <div key={article.articleId}>
                <strong>{article.articleName}</strong>: {article.matchedObjects}
objekt matchade
                <ul>
                  {article.objects.map(obj => (
```

```
                    <li key={obj.id}>{obj.name}</li>
                  ))}
                </ul>
              </div>
            ))}
          </div>
        )}
      </div>
    </div>
  );
};
```

5.4 Fas 3: Automatisk Orderbyggare

Steg 3.1: Service för Orderbyggare

Fil: `server/services/orderBuilderService.ts` (ny fil)

Syfte: Bygg komplett orderunderlag genom att aggregera information från klusterhierarkin, artiklar, prissystem och resurser.

```
import { ClusterMetadataService } from './clusterMetadataService';
import { db } from '../db';

export class OrderBuilderService {
  private metadataService = new ClusterMetadataService();

  /**
   * Bygg komplett orderunderlag för en specifik uppgift
   */
  async buildOrderPackage(taskId: string): Promise<OrderPackage> {
    // Hämta uppgift
    const task = await db.select()
      .from(orders)
      .where(eq(orders.id, taskId))
      .limit(1);

    if (!task.length) {
      throw new Error('Task not found');
    }

    const taskData = task[0];

    // Hämta relaterade entiteter
    const [object, article, customer] = await Promise.all([
      this.getObject(taskData.objectId),
      this.getArticle(taskData.articleId),
      this.getCustomer(taskData.customerId),
    ]);

    // Hämta kompilerad metadata från klusterhierarkin
    const metadata = await this.metadataService.compileMetadata(object.clusterId);

    // Bygg orderpaket med all information
    const orderPackage: OrderPackage = {
      // VAR?
      location: {
        address: metadata.address?.value,
        coordinates: metadata.coordinates?.value,
        cluster: await this.getClusterPath(object.clusterId),
      },

      // TILLGÅNG?
      access: {
        accessCode: metadata.access_code?.value,
        keyLocation: metadata.key_location?.value,
        specialInstructions: metadata.access_instructions?.value,
      },

      // HITTA FRAM?
      navigation: {
        photos: metadata.photos?.value || [],
        landmarks: metadata.landmarks?.value,
        parkingInfo: metadata.parking_info?.value,
      },

      // VAD SKA GÖRAS?
      work: {
        article: {
          id: article.id,
          name: article.name,
          description: article.description,
        },
```

```
        object: {
          id: object.id,
          name: object.name,
          type: object.objectType,
          quantity: object.properties?.quantity,
          specificProperties: object.properties,
        },
      },

      // TID & RESURS?
      scheduling: {
        productionTime: article.standardProductionTime,
        desiredDelivery: taskData.desiredDeliveryDate,
        assignedResource: taskData.resourceId ? await this.getRe-
source(taskData.resourceId) : null,
        assignedVehicle: taskData.vehicleId ? await this.getVe-
hicle(taskData.vehicleId) : null,
      },

      // FAKTURERING?
      billing: {
        customer: {
          name: customer.name,
          invoiceEmail: metadata.invoice_email?.value,
          invoiceReference: metadata.invoice_reference?.value,
        },
        contract: metadata.contract_type?.value,
        pricing: {
          salesPrice: taskData.salesPrice,
          cost: taskData.cost,
          priceList: metadata.price_list_id?.value,
        },
      },

      // KONTAKT VID PROBLEM?
      contacts: await this.getContactChain(object.clusterId),
    };

    return orderPackage;
  }

  /**
   * Hämta hela klusterstigen (t.ex. "Castor → BRF Huset → Grytan 5 → Södra")
   */
  async getClusterPath(clusterId: string): Promise<string> {
    const ancestors = await this.metadataService.getAncestors(clusterId);
    return ancestors.map(a => a.name).join(' → ');
  }

  /**
   * Hämta alla kontakter från aktuell nivå upp till root
   */
  async getContactChain(clusterId: string): Promise<Contact[]> {
    const ancestors = await this.metadataService.getAncestors(clusterId);
    const contacts: Contact[] = [];

    for (const ancestor of ancestors.reverse()) { // Root först
      const clusterContacts = await db.select()
        .from(cluster_contacts)
        .where(eq(cluster_contacts.clusterId, ancestor.id));

      contacts.push(...clusterContacts.map(c => ({
        name: c.name,
```

```
        phone: c.phone,
        email: c.email,
        level: ancestor.name,
      }))));
    }

    return contacts;
  }

  // Hjälpfunktioner
  async getObject(id: string) { /* ... */ }
  async getArticle(id: string) { /* ... */ }
  async getCustomer(id: string) { /* ... */ }
  async getResource(id: string) { /* ... */ }
  async getVehicle(id: string) { /* ... */ }
}

interface OrderPackage {
  location: {
    address: string;
    coordinates: { lat: number; lng: number };
    cluster: string;
  };
  access: {
    accessCode?: string;
    keyLocation?: string;
    specialInstructions?: string;
  };
  navigation: {
    photos: string[];
    landmarks?: string;
    parkingInfo?: string;
  };
  work: {
    article: { id: string; name: string; description: string };
    object: { id: string; name: string; type: string; quantity: number; specificProp-
erties: any };
  };
  scheduling: {
    productionTime: number;
    desiredDelivery: Date;
    assignedResource: any;
    assignedVehicle: any;
  };
  billing: {
    customer: { name: string; invoiceEmail: string; invoiceReference: string };
    contract: string;
    pricing: { salesPrice: number; cost: number; priceList: string };
  };
  contacts: Contact[];
}

interface Contact {
  name: string;
  phone: string;
  email: string;
  level: string; // Vilken klusternivå kontakten är från
}
```

Steg 3.2: API för Orderbyggare

Fil: `server/routes/orders.ts` (uppdatera befintlig)

```
// Lägg till endpoint i befintlig orders router

router.get('/:id/package', async (req, res) => {
  const { id } = req.params;
  const service = new OrderBuilderService();

  try {
    const orderPackage = await service.buildOrderPackage(id);
    res.json(orderPackage);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});
```

Steg 3.3: Mobil UI för Orderpaket

Fil: `client/src/mobile/OrderPackageView.tsx` (ny komponent)

Funktionalitet: Visa komplett orderpaket för fältarbetare med all information de behöver.

```
import React, { useEffect, useState } from 'react';
import { MapPin, Key, Camera, Wrench, Clock, CreditCard, Phone } from 'lucide-react';

export const OrderPackageView: React.FC<{ orderId: string }> = ({ orderId }) => {
  const [pkg, setPkg] = useState(null);

  useEffect(() => {
    fetch(`/api/orders/${orderId}/package`)
      .then(res => res.json())
      .then(data => setPkg(data));
  }, [orderId]);

  if (!pkg) return <div>Laddar...</div>;

  return (
    <div className="order-package p-4 space-y-6">
      <h1 className="text-2xl font-bold">Orderunderlag #{orderId}</h1>

      {/* VAR? */}
      <section className="border p-4 rounded">
        <h2 className="flex items-center gap-2 text-xl font-semibold">
          <MapPin /> Var?
        </h2>
        <p className="text-lg">{pkg.location.address}</p>
        <p className="text-sm text-gray-600">{pkg.location.cluster}</p>
        <a
          href={`https://maps.google.com/?q=${pkg.location.coordinates.lat},${pkg.loca
tion.coordinates.lng}`}
          target="_blank"
          className="text-blue-500 underline"
        >
          Öppna i Google Maps
        </a>
      </section>

      {/* TILLGÅNG? */}
      {pkg.access.accessCode && (
        <section className="border p-4 rounded">
          <h2 className="flex items-center gap-2 text-xl font-semibold">
            <Key /> Tillgång
          </h2>
          <p><strong>Nyckelkod:</strong> {pkg.access.accessCode}</p>
          {pkg.access.keyLocation && <p><strong>Nyckel:</strong>
{pkg.access.keyLocation}</p>}
          {pkg.access.specialInstructions && (
            <p className="text-sm text-gray-600">{pkg.access.specialInstructions}</p>
          )}
        </section>
      )}

      {/* HITTA FRAM? */}
      {pkg.navigation.photos.length > 0 && (
        <section className="border p-4 rounded">
          <h2 className="flex items-center gap-2 text-xl font-semibold">
            <Camera /> Hitta fram
          </h2>
          <div className="grid grid-cols-2 gap-2">
            {pkg.navigation.photos.map((photo, idx) => (
              <img key={idx} src={photo} alt={`Navigation ${idx + 1}`} className="roun
ded" />
            ))}
          </div>
```

```
          {pkg.navigation.parkingInfo && (
            <p className="mt-2 text-sm"><strong>Parkering:</strong>
{pkg.navigation.parkingInfo}</p>
          )}
        </section>
      )}

      {/* VAD SKA GÖRAS? */}
      <section className="border p-4 rounded bg-blue-50">
        <h2 className="flex items-center gap-2 text-xl font-semibold">
          <Wrench /> Vad ska göras?
        </h2>
        <p className="text-lg font-bold">{pkg.work.article.name}</p>
        <p className="text-gray-600">{pkg.work.article.description}</p>
        <div className="mt-2">
          <p><strong>Objekt:</strong> {pkg.work.object.name}</p>
          <p><strong>Typ:</strong> {pkg.work.object.type}</p>
          {pkg.work.object.quantity && <p><strong>Antal:</strong>
{pkg.work.object.quantity}</p>}
        </div>
      </section>

      {/* TID & RESURS */}
      <section className="border p-4 rounded">
        <h2 className="flex items-center gap-2 text-xl font-semibold">
          <Clock /> Tid & Resurs
        </h2>
        <p><strong>Beräknad tid:</strong> {pkg.scheduling.productionTime} minuter</p>
        <p><strong>Önskad leverans:</strong> {new
Date(pkg.scheduling.desiredDelivery).toLocaleDateString('sv-SE')}</p>
        {pkg.scheduling.assignedResource && (
          <p><strong>Utförare:</strong> {pkg.scheduling.assignedResource.name}</p>
        )}
        {pkg.scheduling.assignedVehicle && (
          <p><strong>Fordon:</strong>
{pkg.scheduling.assignedVehicle.registrationNumber}</p>
        )}
      </section>

      {/* FAKTURERING */}
      <section className="border p-4 rounded">
        <h2 className="flex items-center gap-2 text-xl font-semibold">
          <CreditCard /> Fakturering
        </h2>
        <p><strong>Kund:</strong> {pkg.billing.customer.name}</p>
        <p><strong>Avtal:</strong> {pkg.billing.contract}</p>
        <p><strong>Pris:</strong> {pkg.billing.pricing.salesPrice} kr</p>
      </section>

      {/* KONTAKT */}
      <section className="border p-4 rounded">
        <h2 className="flex items-center gap-2 text-xl font-semibold">
          <Phone /> Kontakt vid problem
        </h2>
        {pkg.contacts.map((contact, idx) => (
          <div key={idx} className="mb-2">
            <p className="font-semibold">{contact.name} ({contact.level})</p>
            <a href={`tel:${contact.phone}`} className="text-blue-500">{contact.phone}
</a>
            {contact.email && (
              <a href={`mailto:${contact.email}`} className="text-blue-500 ml-4">{con-
tact.email}</a>
            )}
```

```
            </div>
          ))}
        </section>
      </div>
    );
  };
```

5.5 Fas 4: Integration och Förbättringar

Steg 4.1: Kundkoppling och Aviseringssystem

Databas:

```
// Uppdatera customers tabell
export const customers = pgTable('customers', {
  // ... befintliga kolumner
  primaryClusterId: uuid('primary_cluster_id').references(() => clusterHierarchy.id),
});

// Ny tabell: cluster_contacts
export const clusterContacts = pgTable('cluster_contacts', {
  id: uuid('id').primaryKey().defaultRandom(),
  clusterId: uuid('cluster_id').references(() => clusterHierarchy.id).notNull(),
  contactType: pgEnum('contact_type', ['primary', 'technical', 'billing', 'emergency']
)('contact_type').notNull(),
  name: varchar('name', { length: 255 }).notNull(),
  phone: varchar('phone', { length: 50 }),
  email: varchar('email', { length: 255 }),
  notifyOnOrderCreate: boolean('notify_on_order_create').default(false),
  notifyOnOrderComplete: boolean('notify_on_order_complete').default(false),
});

// Ny tabell: notifications_log
export const notificationsLog = pgTable('notifications_log', {
  id: uuid('id').primaryKey().defaultRandom(),
  clusterId: uuid('cluster_id').references(() => clusterHierarchy.id),
  contactId: uuid('contact_id').references(() => clusterContacts.id),
  notificationType: pgEnum('notification_type', [
    'order_confirmation',
    'work_started',
    'work_completed',
    'issue_reported'
  ])('notification_type').notNull(),
  orderId: uuid('order_id').references(() => orders.id),
  sentAt: timestamp('sent_at').defaultNow(),
  deliveryStatus: pgEnum('delivery_status', ['sent', 'delivered', 'failed'])('deliv-
ery_status'),
});
```

Service:

```
// server/services/notificationService.ts
export class NotificationService {
  async sendOrderConfirmation(orderId: string) {
    const order = await this.getOrder(orderId);
    const contacts = await this.getRelevantContacts(order.sourceClusterId, 'or-
der_confirmation');

    for (const contact of contacts) {
      if (contact.email && contact.notifyOnOrderCreate) {
        await this.sendEmail(
          contact.email,
          'Orderbekräftelse',
          this.generateOrderConfirmationEmail(order)
        );

        await this.logNotification(order.sourceClusterId, contact.id, 'or-
der_confirmation', orderId);
      }
    }
  }

  async getRelevantContacts(clusterId: string, eventType: string): Promise<Contact[]>
{
    // Hämta kontakter från aktuell nivå och uppåt
    const ancestors = await new ClusterMetadataService().getAncestors(clusterId);
    const allContacts = [];

    for (const ancestor of ancestors) {
      const contacts = await db.select()
        .from(clusterContacts)
        .where(eq(clusterContacts.clusterId, ancestor.id));
      allContacts.push(...contacts);
    }

    return allContacts;
  }
}
```

Steg 4.2: Prissystemintegration med Kluster

Uppdatera prislistor:

```
// Lägg till i priceLists tabell
export const priceLists = pgTable('price_lists', {
  // ... befintliga kolumner
  clusterId: uuid('cluster_id').references(() => clusterHierarchy.id), // null =
generell prislista
});

export const discountLetters = pgTable('discount_letters', {
  // ... befintliga kolumner
  clusterId: uuid('cluster_id').references(() => clusterHierarchy.id), // null =
generellt rabattbrev
});
```

Uppdatera OrderConceptService.getPricing():

```
async getPricing(articleId: string, priceListId: string, customerId: string,
clusterId: string) {
  // 1. Sök efter rabattbrev kopplat till klustret eller föräldrar
  const ancestors = await this.metadataService.getAncestors(clusterId);
  const clusterIds = [clusterId, ...ancestors.map(a => a.id)];

  // Leta efter rabattbrev (högsta prioritet)
  const discountLetter = await db.select()
    .from(discountLetters)
    .where(
      and(
        eq(discountLetters.customerId, customerId),
        inArray(discountLetters.clusterId, clusterIds)
      )
    )
    .limit(1);

  if (discountLetter.length) {
    // Använd rabattbrevspriser
  }

  // 2. Sök efter kundspecifik prislista
  const customerPriceList = await db.select()
    .from(priceLists)
    .where(
      and(
        eq(priceLists.customerId, customerId),
        inArray(priceLists.clusterId, clusterIds)
      )
    )
    .limit(1);

  if (customerPriceList.length) {
    // Använd kundspecifik prislista
  }

  // 3. Fallback till generell prislista
  const generalPriceList = await db.select()
    .from(priceLists)
    .where(isNull(priceLists.clusterId))
    .limit(1);

  // Returnera priser
}
```

Steg 4.3: “Snöret” - Tre-vägs Planeringsoptimering

Databas:

```
// Uppdatera orders/tasks tabell
export const tasks = pgTable('tasks', {
  // ... befintliga kolumner

  // Objektets/kundens tider
  customerTimeWindow: jsonb('customer_time_window'), // { start: "08:00", end:
"12:00" }
  customerSchedule: jsonb('customer_schedule'), // Återkommande schema

  // Uppgiftens tider
  desiredDeliveryDate: date('desired_delivery_date'),
  deadline: timestamp('deadline'),
  productionTimePlanned: integer('production_time_planned'), // minuter
  productionTimeActual: integer('production_time_actual'),

  // Utförarens tider
  resourceId: uuid('resource_id').references(() => resources.id),
  plannedStart: timestamp('planned_start'),
  plannedEnd: timestamp('planned_end'),
  actualStart: timestamp('actual_start'),
  actualEnd: timestamp('actual_end'),
  travelTimePlanned: integer('travel_time_planned'), // minuter
  travelTimeActual: integer('travel_time_actual'),
  preparationTime: integer('preparation_time'), // minuter

  taskType: pgEnum('task_type', ['production', 'preparation', 'travel', 'rest', 'ad-
min'])('task_type'),
});

// Utöka resources tabell
export const resources = pgTable('resources', {
  // ... befintliga kolumner

  // Tillgänglighetstider
  availability: jsonb('availability'), // { monday: { start: "07:00", end:
"16:00" }, ... }
  breakTimes: jsonb('break_times'), // { lunch: { start: "12:00", duration:
30 }, ... }
});
```

Service:

```
// server/services/stringPlanningService.ts
export class StringPlanningService {
  /**
   * Optimera "snöret" för en resurs under en vecka
   * Tar hänsyn till tre tidsintressen:
   * 1. Objektets/kundens tider
   * 2. Uppgiftens tider
   * 3. Utförarens tider
   */
  async optimizeString(resourceId: string, weekNumber: number, year: number):
Promise<OptimizedString> {
    // 1. Hämta alla uppgifter för resursen under veckan
    const tasks = await this.getTasksForResourceWeek(resourceId, weekNumber, year);

    // 2. Hämta resursens tillgänglighet
    const resource = await this.getResource(resourceId);
    const availability = this.parseAvailability(resource.availability, weekNumber, yea
r);

    // 3. För varje uppgift, beräkna möjliga tidsfönster
    const taskWindows = tasks.map(task => {
      return {
        task,
        customerWindow: this.parseCustomerWindow(task.customerTimeWindow),
        taskDeadline: task.deadline,
        resourceAvailability: availability,
        possibleSlots: this.findPossibleSlots(task, availability),
      };
    });

    // 4. Använd optimeringsalgoritm (eller extern Unicorn Optimization Service)
    const optimizedSchedule = await this.runOptimization(taskWindows, resource);

    // 5. Beräkna "snöret" - sekvens av tider
    const string = this.buildString(optimizedSchedule, resource);

    // 6. Beräkna nyckeltal
    const metrics = this.calculateMetrics(string, resource, weekNumber, year);

    return {
      resourceId,
      week: weekNumber,
      year,
      string,
      metrics,
    };
  }

  /**
   * Bygg "snöret" - sekvens av uppgifter med tidstyper
   */
  buildString(schedule: any[], resource: any): StringSegment[] {
    const string: StringSegment[] = [];

    for (let i = 0; i < schedule.length; i++) {
      const task = schedule[i];

      // Lägg till förberedelse om behövs
      if (task.preparationTime > 0) {
        string.push({
          type: 'preparation',
          taskId: task.id,
```

```
          start: task.plannedStart,
          duration: task.preparationTime,
          status: 'planned',
        });
      }

      // Lägg till resa till arbetsplatsen (om inte första eller om förra var på annan
plats)
      if (i > 0 || task.location !== resource.homeLocation) {
        const travelTime = await this.calculateTravelTime(
          i === 0 ? resource.homeLocation : schedule[i-1].location,
          task.location
        );

        string.push({
          type: 'travel',
          from: i === 0 ? 'hem' : schedule[i-1].location,
          to: task.location,
          start: task.plannedStart - travelTime * 60000, // Convert to ms
          duration: travelTime,
          status: 'planned',
        });
      }

      // Lägg till själva produktionen
      string.push({
        type: 'production',
        taskId: task.id,
        start: task.plannedStart,
        duration: task.productionTimePlanned,
        status: task.status,
      });
    }

    return string;
  }

  /**
   * Beräkna nyckeltal för "snöret"
   */
  calculateMetrics(string: StringSegment[], resource: any, week: number, year:
number): StringMetrics {
    const totalTime = this.getTotalWorkHours(week, year);

    const productionTime = string
      .filter(s => s.type === 'production')
      .reduce((sum, s) => sum + s.duration, 0);

    const travelTime = string
      .filter(s => s.type === 'travel')
      .reduce((sum, s) => sum + s.duration, 0);

    const preparationTime = string
      .filter(s => s.type === 'preparation')
      .reduce((sum, s) => sum + s.duration, 0);

    const totalPlannedTime = productionTime + travelTime + preparationTime;

    return {
      weekGoalPercent: (productionTime / (totalTime * 0.7)) * 100, // Antag 70% av
tiden ska vara produktion
      totalCalculatedTime: totalTime,
      productionTime: productionTime / 60, // Convert to hours
```

```
      travelTime: travelTime / 60,
      preparationTime: preparationTime / 60,
      utilization: (totalPlannedTime / (totalTime * 60)) * 100,
    };
  }
}

interface StringSegment {
  type: 'production' | 'preparation' | 'travel' | 'rest' | 'admin';
  taskId?: string;
  from?: string;
  to?: string;
  start: Date;
  duration: number; // minuter
  status: string;
}

interface StringMetrics {
  weekGoalPercent: number;
  totalCalculatedTime: number; // timmar
  productionTime: number; // timmar
  travelTime: number; // timmar
  preparationTime: number; // timmar
  utilization: number; // procent
}

interface OptimizedString {
  resourceId: string;
  week: number;
  year: number;
  string: StringSegment[];
  metrics: StringMetrics;
}
```

Frontend - “Snöret” Visualisering:

```
// client/src/components/StringVisualization.tsx
export const StringVisualization: React.FC<{ resourceId: string, week: number, year: n
umber }> = ({ resourceId, week, year }) => {
  const [stringData, setStringData] = useState(null);

  useEffect(() => {
    fetch(`/api/planning/string/${resourceId}/${week}/${year}`)
      .then(res => res.json())
      .then(data => setStringData(data));
  }, [resourceId, week, year]);

  if (!stringData) return <div>Laddar...</div>;

  return (
    <div>
      <h2>"Snöret" för {resourceId}, vecka {week}</h2>

      {/* Nyckeltal */}
      <div className="metrics grid grid-cols-5 gap-4 mb-4">
        <div className="metric">
          <span className="label">Veckomål:</span>
          <span className="value">{stringData.metrics.weekGoalPercent.toFixed(1)}%</
span>
        </div>
        <div className="metric">
          <span className="label">Total tid:</span>
          <span className="value">{stringData.metrics.totalCalculatedTime}h</span>
        </div>
        <div className="metric">
          <span className="label">Produktion:</span>
          <span className="value">{stringData.metrics.productionTime.toFixed(1)}h</
span>
        </div>
        <div className="metric">
          <span className="label">Restid:</span>
          <span className="value">{stringData.metrics.travelTime.toFixed(1)}h</span>
        </div>
        <div className="metric">
          <span className="label">Beläggning:</span>
          <span className="value">{stringData.metrics.utilization.toFixed(1)}%</span>
        </div>
      </div>

      {/* Snöret - visuell tidslinje */}
      <div className="string-timeline">
        {stringData.string.map((segment, idx) => (
          <div
            key={idx}
            className={`segment segment-${segment.type}`}
            style={{
              width: `${(segment.duration / stringData.metrics.totalCalculatedTime) *
100}%`
            }}
          >
            <span className="type">{segment.type}</span>
            <span className="duration">{segment.duration}min</span>
          </div>
        ))}
      </div>

      {/* Detaljerad lista */}
      <table>
```

```
        <thead>
          <tr>
            <th>Status</th>
            <th>Uppdrag</th>
            <th>Typ</th>
            <th>Tid</th>
            <th>Plats</th>
            <th>Plan</th>
            <th>Utfall</th>
          </tr>
        </thead>
        <tbody>
          {stringData.string.map((segment, idx) => (
            <tr key={idx}>
              <td>{segment.status}</td>
              <td>{segment.taskId || '-'}</td>
              <td>{segment.type}</td>
              <td>{new Date(segment.start).toLocaleTimeString('sv-SE')}</td>
              <td>{segment.to || '-'}</td>
              <td>{segment.duration}min</td>
              <td>-</td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  );
};
```

Steg 4.4: Felanmälningar Kopplade till Kluster

Databas:

```
export const issueReports = pgTable('issue_reports', {
  id: uuid('id').primaryKey().defaultRandom(),
  clusterId: uuid('cluster_id').references(() => clusterHierarchy.id).notNull(),
  objectId: uuid('object_id').references(() => objects.id), // Specifikt objekt om
tillämpligt
  reportedBy: varchar('reported_by', { length: 255 }),
  contactInfo: varchar('contact_info', { length: 255 }),
  issueType: varchar('issue_type', { length: 100 }), // t.ex. 'damaged', 'missing',
'access_problem'
  description: text('description'),
  severity: pgEnum('severity', ['low', 'medium', 'high', 'critical'])('severity'),
  status: pgEnum('issue_status', ['reported', 'acknowledged', 'in_progress', 're-
solved'])('issue_status').default('reported'),
  createdAt: timestamp('created_at').defaultNow(),
  resolvedAt: timestamp('resolved_at'),
  generatedTaskId: uuid('generated_task_id').references(() => orders.id), // Automat-
iskt genererad uppgift
});
```

Service:

```
// server/services/issueReportService.ts
export class IssueReportService {
  async createIssueReport(data: IssueReportData): Promise<IssueReport> {
    // Skapa felanmälan
    const issue = await db.insert(issueReports).values(data).returning();

    // Skicka notifikation till relevanta kontakter
    await this.notifyContacts(issue[0]);

    // Om allvarligt, skapa automatiskt en uppgift
    if (issue[0].severity === 'high' || issue[0].severity === 'critical') {
      const taskId = await this.generateRepairTask(issue[0]);
      await db.update(issueReports)
        .set({ generatedTaskId: taskId })
        .where(eq(issueReports.id, issue[0].id));
    }

    return issue[0];
  }

  async generateRepairTask(issue: IssueReport): Promise<string> {
    // Hitta lämplig reparationsartikel baserat på issueType
    const repairArticle = await this.findRepairArticle(issue.issueType);

    // Hämta klustermetadata
    const metadata = await new ClusterMetadataSer-
vice().compileMetadata(issue.clusterId);

    // Skapa uppgift
    const task = await db.insert(orders).values({
      articleId: repairArticle.id,
      objectId: issue.objectId,
      sourceClusterId: issue.clusterId,
      description: `Felanmälan: ${issue.description}`,
      priority: issue.severity === 'critical' ? 'urgent' : 'high',
      status: 'pending',
    }).returning();

    return task[0].id;
  }

  async searchIssuesInCluster(clusterId: string, filters: any):
Promise<IssueReport[]> {
    // Sök felanmälningar i kluster och dess descendants
    const descendants = await new ClusterMetadataService().getDescendants(clusterId);
    const clusterIds = [clusterId, ...descendants.map(d => d.id)];

    const issues = await db.select()
      .from(issueReports)
      .where(
        and(
          inArray(issueReports.clusterId, clusterIds),
          // ... andra filter
        )
      );

    return issues;
  }
}
```

5.6 Sammanfattning av Implementationsprioritet

Fas 1: KRITISK (Vecka 1-2)

- ✅ Klusterhierarki databas
- ✅ Metadatapropagering
- ✅ API för kluster
- ✅ ClusterMetadataService

Fas 2: KRITISK (Vecka 3-4)

- ✅ Artikelfasthakningar
- ✅ Orderkoncept databas
- ✅ OrderConceptService
- ✅ Förhandsgranskning och exekvering

Fas 3: HÖG (Vecka 5)

- ✅ OrderBuilderService
- ✅ API för orderpaket
- ✅ Mobil UI för orderpaket

Fas 4: MEDEL-LÅG (Vecka 6-8)

- ✅ Kundkoppling och avisering
- ✅ Prissystemintegration
- ✅ "Snöret" - planeringsoptimering
- ✅ Felanmälningar

---

6. Viktiga Principer och Regler

6.1 Designprinciper

Princip 1: Information Registreras på Rätt Nivå

> "Registrera EN GÅNG – Använd ÖVERALLT"

Regel:
- Identifiera vilken nivå i klusterhierarkin som informationen naturligt hör till
- Registrera informationen på den nivån
- Låt systemet propagera informationen nedåt automatiskt

Exempel:
- Fakturamejl: Registreras på koncern-nivå → alla underliggande enheter ärver
- Nyckelkod: Registreras på fastighet-nivå → alla rum och objekt i fastigheten ärver
- Skadat objekt: Registreras på objekt-nivå → specifikt för det objektet

Princip 2: Fallande Information som Tyngdkraft

> "Ändra EN GÅNG – Uppdatera ÖVERALLT"

Regel:
- Information som registreras högre upp i hierarkin "faller nedåt"
- Vid ändring på högre nivå, uppdateras automatiskt alla undernivåer
- Ingen manuell synkronisering behövs

Implementation:
- Använd `compileMetadata()` för att hämta all ärvd metadata
- Spara inte kopierad metadata på varje nivå (referens istället)
- Vid uppdatering, propagera till alla descendants

Princip 3: Möjlighet till Överskridning

> "Huvudregeln gäller, men undantag är möjliga"

Regel:
- Information kan överskridas på lägre nivå om `canOverride = true`
- Prioritet: egen nivå > förälder > farförälder > ... > root
- Spärrmekanism möjliggör specialfall

Exempel:
- Castor: Prislista 120 (gäller alla)
- Fastighet Grytan 5: Prislista 125 (specialavtal) → **överskriver**
- Alla objekt under Grytan 5: använder Prislista 125

Princip 4: Intelligent Artikelmatchning

> "Artiklar hittar sin plats automatiskt"

Regel:
- Artiklar definierar sina fasthakningstyper
- Systemet matchar artiklar till objekt baserat på typ
- Orderkoncept traverserar klustret och skapar uppgifter där matchningar finns

Implementation:
- Använd `article_hooks` för att definiera matchningsregler
- Implementera `findMatchingObjects()` för intelligent sökning
- Stöd för flera fasthakningar per artikel

Princip 5: Fullständigt Orderunderlag

> "Fältarbetaren ska ha allt hen behöver – automatiskt"

Regel:
- Varje order byggs genom att aggregera information från hela klusterhierarkin
- Orderunderlaget ska innehålla: VAR, TILLGÅNG, HITTA FRAM, VAD, TID & RESURS, FAKTURERING, KONTAKT
- Ingen manuell insamling av information

Implementation:
- Använd `OrderBuilderService.buildOrderPackage()`
- Samla metadata från alla nivåer
- Presentera i strukturerad, lättläst format för fältarbetare

Princip 6: Tre Tidsintressen i Balans

> "Kunden, uppgiften och utföraren måste mötas"

Regel:
- Planering ska ta hänsyn till:
1. Kundens/objektets tider (tidsfönster, schema)
2. Uppgiftens tider (deadline, produktionstid)

3. Utförarens tider (tillgänglighet, restid, raster)
- “Snöret” visualiserar hela veckan

Implementation:
- Implementera `StringPlanningService`
- Optimeringsalgoritm som balanserar alla tre
- Nyckeltal för uppföljning

6.2 Affärsregler

Regel 1: Klustertyper och Hierarki

Tillåtna typer:
- `organization` (koncern) - Nivå 0 eller 1
- `legal_entity` (juridisk person) - Nivå 1 eller 2
- `property` (fastighet) - Nivå 2 eller 3
- `area` (område) - Nivå 2, 3 eller 4
- `room` (rum) - Nivå 3, 4 eller 5
- `object` (objekt) - Nivå 4, 5 eller 6
- `unit` (individuell enhet) - Nivå 5, 6 eller 7

Begränsningar:
- Maxdjup: 7 nivåer (konfigurerbart)
- Varje kluster måste ha exakt en förälder (utom root)
- Cykliska referenser är förbjudna

Regel 2: Metadatatyper

Fast information (`fixed`):
- Kan INTE uppdateras automatiskt
- Stannar på den nivå där den registrerades
- Synlig för alla undernivåer (referens)
- Exempel: nyckelkod, specifik skada

Fallande information (`cascading`):
- Ärvs automatiskt till alla undernivåer
- Kan överskridas om `canOverride = true`
- Ändras inte över tid (statisk konfiguration)
- Exempel: avtal, generella instruktioner

Dynamisk information (`dynamic`):
- Ärvs automatiskt till alla undernivåer
- Kan överskridas om `canOverride = true`
- **Kan uppdateras över tid**
- Vid uppdatering, propagerar till alla undernivåer som inte överskrivit
- Exempel: kontaktuppgifter, status

Regel 3: Orderkoncept-exekvering

Sekvens:
1. Validera orderkoncept (targetClusterId finns, artiklar finns)
2. Hämta alla artiklar i orderkonceptet
3. För varje artikel:
- Hämta fasthakningar
- Hitta matchande objekt i målklustret och descendants

- Skapa uppgift för varje matchning
- Om artikel har beroendeartiklar, skapa dem
4. Uppdatera orderkoncept status till `active`
5. Skicka aviseringar till kontakter

Villkor för matchning:
- Objektets `objectType` måste matcha artikelns `hookType`
- Objektet måste finnas i målklustret eller någon av dess descendants
- Objektet får inte vara inaktivt/borttaget

Regel 4: Prisprioritering

Ordning (högsta prioritet först):
1. **Rabattbrev** kopplat till klustret eller föräldrar
2. **Kundspecifik prislista** kopplad till klustret eller föräldrar
3. **Generell prislista** (ingen cluster_id)

Vid flera träffar på samma nivå:
- Välj den som är kopplad till lägsta klusternivå (närmast objektet)

Exempel:

```
Rabattbrev på BRF Huset (nivå 2)
Kundspecifik prislista på Castor (nivå 1)
Generell prislista (ingen nivå)

→ Rabattbrevet på BRF Huset används (högsta prioritet)
```

Regel 5: Avisering

När aviseringar skickas:
- **Orderbekräftelse:** När orderkoncept exekveras och uppgifter skapas
- **Arbete påbörjat:** När fältarbetare markerar uppgift som `in_progress`
- **Arbete slutfört:** När fältarbetare markerar uppgift som `completed`
- **Problem rapporterat:** När felanmälan skapas

Vilka kontakter notifieras:
- Hämta alla kontakter från objektets klusternivå och uppåt
- Filtrera baserat på kontakttyp och notifikationsinställningar
- Skicka via e-post och/eller SMS (konfigurerbart)

Logga alla aviseringar:
- Spara i `notifications_log`
- Spåra leveransstatus (`sent`, `delivered`, `failed`)

Regel 6: “Snöret” - Tidsberäkning

Tidstyper som ingår:
- **Egentid:** Resursens personliga tid (vila, ledighet)
- **Förberedelsetid:** Förberedelse innan produktion
- **Restid:** Till/från arbetsplats
- **Produktionstid:** Själva arbetet
- **Rast:** Lunch, pauser
- **Administration:** Rapportering, etc.

Beräkning av beläggning:

```
Beläggning (%) = (Produktionstid + Restid + Förberedelsetid) / Total arbetstid * 100
```

Målsättning:
- Produktion bör vara minst 70% av total arbetstid
- Restid bör hållas under 20%
- Resterande för rast och administration

6.3 Edge Cases och Felhantering

Edge Case 1: Cyklisk Klusterhierarki

Problem: Om ett kluster pekar på sin egen förälder eller en förälder pekar nedåt, skapas en cykel.

Lösning:
- Vid skapande/uppdatering av kluster, kontrollera att `parentId` inte skapar cykel
- Implementera cykeldetektion i `ClusterMetadataService.getAncestors()`
- Kasta fel om cykel upptäcks

```
async validateNoCycle(clusterId: string, newParentId: string): Promise<boolean> {
  const ancestors = await this.getAncestors(newParentId);
  return !ancestors.some(a => a.id === clusterId);
}
```

Edge Case 2: Orderkoncept Utan Matchningar

Problem: Ett orderkoncept exekveras men hittar inga matchande objekt.

Lösning:
- `executeOrderConcept()` returnerar resultat med `tasksCreated = 0`
- Logga varning
- Notifiera användaren att inga uppgifter skapades
- Markera orderkoncept som `completed` ändå (för historik)

Edge Case 3: Metadata Konflikt (Flera Överskrivningar)

Problem: Både nivå 2 och nivå 3 överskriver samma metadata från nivå 1.

Lösning:
- Prioritet: närmaste nivån vinner
- I `compileMetadata()`, iterera från root till leaf
- Senaste värdet (lägsta nivån) används

```
// Nivå 1 (Castor): invoice_email = "ekonomi@castor.se"
// Nivå 2 (BRF Huset): invoice_email = "faktura@brfhuset.se"
// Nivå 3 (Grytan 5): invoice_email = "grytan5@example.com"

// Resultat för Grytan 5: "grytan5@example.com" (närmaste nivån)
```

Edge Case 4: Borttagen Artikel i Aktivt Orderkoncept

Problem: En artikel tas bort från systemet men är fortfarande kopplad till ett aktivt orderkoncept.

Lösning:
- Soft delete på artiklar (flagga som `deleted` istället för att radera)

- Vid exekvering, hoppa över borttagna artiklar
- Logga varning om borttagen artikel påträffas
- Notifiera användaren

Edge Case 5: Objekt Flyttas mellan Kluster

Problem: Ett objekt flyttas från ett kluster till ett annat, men har redan uppgifter kopplade till sig.

Lösning:
- Tillåt flytt av objekt genom att uppdatera `clusterId`
- Befintliga uppgifter behåller sin `sourceClusterId` (historik)
- Nya uppgifter använder nya `clusterId`
- Kompilerad metadata i befintliga uppgifter uppdateras inte (fryst vid skapande)

Edge Case 6: Prislista Saknas

Problem: Ingen prislista hittas för artikel, varken rabattbrev, kundspecifik eller generell.

Lösning:
- Kasta fel vid orderskapande om ingen prislista hittas
- Kräv att en generell prislista alltid finns (systemkrav)
- Vid import/konfiguration, validera att generell prislista existerar

Edge Case 7: Beroendeartiklar i Kedja

Problem: Artikel A har beroendeartiklar B och C. B har i sin tur beroendeartiklar D och E.

Lösning:
- Stöd endast ett nivå av beroenden (B och C skapas, men inte D och E)
- Eller: Implementera rekursiv beroendekedjor med maxdjup (t.ex. 3 nivåer)
- Dokumentera begränsningen tydligt

Rekommendation: Ett nivå för första implementationen, utöka senare vid behov.

Edge Case 8: Snöret - Överlappande Tider

Problem: Två uppgifter schemaläggs på samma resurs vid samma tid.

Lösning:
- Optimeringsalgoritmen måste garantera icke-överlappande tider
- Validera schema före sparande
- Kasta fel om överlappning upptäcks
- UI visar konflikt och föreslår alternativ

```
function validateNoOverlap(tasks: Task[]): boolean {
  const sorted = tasks.sort((a, b) => a.plannedStart - b.plannedStart);

  for (let i = 0; i < sorted.length - 1; i++) {
    const current = sorted[i];
    const next = sorted[i + 1];

    if (current.plannedEnd > next.plannedStart) {
      throw new Error(`Overlap detected between task ${current.id} and ${next.id}`);
    }
  }

  return true;
}
```

6.4 Prestandaoptimering

Optimering 1: Caching av Kompilerad Metadata

Problem: `compileMetadata()` gör många databas-queries vid varje anrop.

Lösning:
- Implementera Redis-cache för kompilerad metadata
- Cache-nyckel: `cluster_metadata:{clusterId}`
- TTL: 1 timme (konfigurerbart)
- Invalidera cache vid uppdatering av metadata på klusternivå eller föräldrar

```
async compileMetadata(clusterId: string): Promise<Record<string, any>> {
  // Försök hämta från cache
  const cached = await redis.get(`cluster_metadata:${clusterId}`);
  if (cached) {
    return JSON.parse(cached);
  }

  // Kompilera från databas
  const compiled = await this.compileMetadataFromDB(clusterId);

  // Spara i cache
  await redis.set(`cluster_metadata:${clusterId}`, JSON.stringify(compiled), 'EX', 360
0);

  return compiled;
}
```

Optimering 2: Batch-hämtning av Objekt

Problem: `findMatchingObjects()` kan generera många individuella queries.

Lösning:
- Använd `WHERE IN` för att hämta alla objekt i en query
- Implementera dataloader-mönster för batch-requests

```
// Istället för:
for (const clusterId of clusterIds) {
  const objects = await db.select().from(objects).where(eq(objects.clusterId, cluster-
Id));
}

// Gör:
const allObjects = await db.select().from(objects).where(inArray(objects.clusterId, cl
usterIds));
```

Optimering 3: Index på Databastabeller

Kritiska index:
- `cluster_hierarchy(parent_id)` - För traversering
- `cluster_hierarchy(tenant_id)` - För multi-tenancy
- `cluster_metadata(cluster_id, key)` - För metadata-lookup
- `article_hooks(article_id)` - För artikelmatchning
- `article_hooks(hook_type)` - För typbaserad sökning
- `objects(cluster_id)` - För objekthämtning
- `objects(object_type)` - För typmatchning

- `orders(order_concept_id)` - För uppgiftshämtning
- `orders(source_cluster_id)` - För kluster-baserade rapporter

Optimering 4: Lazy Loading av Descendants

Problem: Att hämta alla descendants för ett stort kluster kan ta lång tid.

Lösning:
- Implementera paginering för descendants
- Hämta endast de nivåer som behövs (t.ex. bara 2 nivåer ned)
- Använd `LIMIT` i rekursiv CTE

```
async getDescendants(clusterId: string, maxDepth: number = 10): Promise<ClusterHier-
archy[]> {
  const result = await db.execute(sql`
    WITH RECURSIVE descendants AS (
      SELECT *, 0 as depth FROM cluster_hierarchy WHERE id = ${clusterId}
      UNION ALL
      SELECT ch.*, d.depth + 1
      FROM cluster_hierarchy ch
      INNER JOIN descendants d ON ch.parent_id = d.id
      WHERE d.depth < ${maxDepth}
    )
    SELECT * FROM descendants ORDER BY level ASC
  `);

  return result.rows;
}
```

Slutsats

Denna systemspecifikation ger Replit 3.0 en komplett bild av:

1. **Nuläget:** Vad som finns idag i Unicorn-systemet
2. **Visionen:** Mats koncept för klusterhantering med nedåtpropagering
3. **Överenskommen Design:** Familjeträdsmetaforen och tre typer av metadata
4. **Gap-analys:** Vad som saknas för att nå målbilden
5. **Implementationsguide:** Konkreta steg, databasscheman, API:er och kod
6. **Principer och Regler:** Designfilosofi, affärsregler, edge cases och optimeringar

Nästa Steg för Replit 3.0:

Börja med **Fas 1** (Klusterhierarki och Metadatapropagering) för att bygga grunden. Detta är det mest kritiska och allt annat bygger på detta. Följ sedan de prioriterade faserna i ordning.

Viktigt att Komma Ihåg:

- **Registrera EN GÅNG – Använd ÖVERALLT**
- **Ändra EN GÅNG – Uppdatera ÖVERALLT**
- **Information faller nedåt som tyngdkraft**
- **Artiklar hittar sin plats automatiskt**
- **Orderunderlaget byggs genom att läsa hela trädet**

Lycka till med implementationen! 🚀

Dokumentslut